



ระบบควบคุมภายในยานยนต์ด้วยเสียง Voice-Controlled In-Vehicle System

จัดทำโดย

คุณานนท์	วันเย็น	66010103
Kunanon	Wanyen	66010103
ปรมัตถ์	ทองดา	66010454
Porramat	Thongda	66010454

รายงานนี้เป็นส่วนหนึ่งของรายวิชาการประยุกต์ทางอิเล็กทรอนิกส์ 2
ภาควิชาวิศวกรรมอิเล็กทรอนิกส์ คณะวิศวกรรมศาสตร์
สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง
ปีการศึกษา พ.ศ. 2568

ระบบควบคุมภายในยานยนต์ด้วยเสียง
Voice-Controlled In-Vehicle System

จัดทำโดย

คุณานนท์	วันเย็น	66010103
Kunanon	Wanyen	66010103
ปรมัตต์	ทองดา	66010454
Porramat	Thongda	66010454

อาจารย์ที่ปรึกษา

ผศ. ดร.สุเมธ วิศยทักษิณ

รายงานนี้เป็นส่วนหนึ่งของรายวิชาการประยุกต์ทางอิเล็กทรอนิกส์ 2
ภาควิชาวิศวกรรมอิเล็กทรอนิกส์ คณะวิศวกรรมศาสตร์
สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง
ปีการศึกษา พ.ศ. 2568

รายงานวิชา	การประยุกต์ทางอิเล็กทรอนิกส์ 2
ปีการศึกษา	2568
ภาควิชา	วิศวกรรมอิเล็กทรอนิกส์
คณะ	วิศวกรรมศาสตร์
มหาวิทยาลัย	สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง
เรื่อง	ระบบควบคุมภายในยานยนต์ด้วยเสียง Voice-Controlled In-Vehicle System
ผู้จัดทำ	นายคุณานต์ วันเย็น รหัสนักศึกษา 66010103 นายปรมัตถ์ ทองดา รหัสนักศึกษา 66010454

รายงานนี้ผ่านการตรวจสอบโดยอาจารย์ที่ปรึกษาแล้ว

ผศ. ดร.สุเมฆ วิศยทักษิณ
อาจารย์ที่ปรึกษา

หัวข้อโครงการ	ระบบควบคุมภายในยานยนต์ด้วยเสียง
นักศึกษา	นายคุณานนต์ วันเย็น รหัสนักศึกษา 66010103 นายปรมัตต์ ทองดา รหัสนักศึกษา 66010454
ปริญญา	วิศวกรรมศาสตร์
ภาควิชา	วิศวกรรมอิเล็กทรอนิกส์
ปีการศึกษา	2568
อาจารย์ที่ปรึกษาโครงการ	ผศ. ดร.สุเมธ วิทยทักษิณ

บทคัดย่อ

โครงการนี้นำเสนอการพัฒนาารบบควบคุมอุปกรณ์ภายในรถยนต์ผ่านคำสั่งเสียง โดยใช้ Raspberry Pi เป็นหน่วยประมวลผลหลักในการรับและประมวลผลคำสั่งเสียงจากผู้ใช้ ระบบรับสัญญาณเสียงผ่านไมโครโฟนและแปลงสัญญาณเสียงเป็นข้อความด้วยเทคโนโลยี Speech Recognition จากนั้นทำการวิเคราะห์ความหมายของคำสั่งเพื่อระบุประเภทของการควบคุมที่ต้องการ เมื่อได้ผลลัพธ์ของคำสั่งแล้ว ระบบจะทำการแปลงคำสั่งให้อยู่ในรูปแบบ CAN message และส่งข้อมูลผ่านเครือข่าย Controller Area Network (CAN Bus) เพื่อควบคุมอุปกรณ์ภายในระบบ เช่น ระบบปรับอากาศ กระบอกไฟฟ้า และไฟภายในรถยนต์ ผลการทดสอบพบว่าระบบสามารถรับและประมวลผลคำสั่งเสียงของผู้ใช้ได้ และสามารถส่ง CAN message เพื่อควบคุมอุปกรณ์ภายในระบบได้ตามที่กำหนด ซึ่งแสดงให้เห็นถึงความเป็นไปได้ในการประยุกต์ใช้เทคโนโลยีการรู้จำเสียงร่วมกับระบบสื่อสาร CAN สำหรับการควบคุมอุปกรณ์ภายในรถยนต์

Project Title	Voice-Controlled In-Vehicle System
Student	Kunanon Wanyen Student ID 66010103 Porrarnat Thongda Student ID 66010454
Degree	Bachelor of Engineering
Program	Electronics Engineering
Year	2025
Project Advisor	Asst. Prof. Sumek Wisayataksin

ABSTRACT

This project presents the development of a voice-controlled system for operating in-vehicle functions using a Raspberry Pi as the main processing unit. The system receives voice input from a microphone and converts the speech signal into text using Speech Recognition technology. The recognized text is further analyzed to determine the user's command and its corresponding intent. After identifying the command, the system converts the instruction into a CAN message and transmits it through the Controller Area Network (CAN Bus) to control various vehicle functions such as the air conditioning system, power windows, and cabin lights. The experimental results show that the system can successfully recognize voice commands and transmit CAN messages to control the target devices as intended. This project demonstrates the feasibility of integrating speech recognition technology with CAN communication to develop a voice-based control system for automotive applications.

กิตติกรรมประกาศ

ข้าพเจ้าขอแสดงความขอบคุณ ผศ. ดร.สุเมธ วิศยทักษิณ อาจารย์ที่ปรึกษาซึ่งได้ให้คำแนะนำและความช่วยเหลือตลอดการทำโครงการนี้ การสนับสนุนและคำปรึกษาของท่านมีคุณค่าต่อความสำเร็จของโครงการนี้เป็นอย่างยิ่ง

ขอขอบคุณรุ่นพี่นักศึกษาระดับบัณฑิตศึกษา รุ่นพี่นักศึกษาระดับปริญญาตรีชั้นปีที่ 4 และเพื่อนนักศึกษาระดับปริญญาตรีชั้นปีที่ 3 ทุกคนที่ช่วยสอนและคอยให้คำแนะนำในการจัดทำโครงการนี้

สุดท้ายนี้ ข้าพเจ้าขอขอบคุณครอบครัวและคนใกล้ชิด ที่ได้ให้กำลังใจและสนับสนุนทางด้านจิตใจตลอดช่วงเวลาที่ข้าพเจ้าทำโครงการนี้จนสำเร็จลุล่วง

ข้าพเจ้าหวังว่าโครงการเล่มนี้จะเป็นประโยชน์ต่อผู้อ่านไม่มากนักน้อย หากมีข้อผิดพลาดประการใด ข้าพเจ้าขออภัยมา ณ ที่นี้ด้วย

คุณานนต์ วันเย็น

ปรมัตถ์ ทองดา

สารบัญ

หน้า

บทคัดย่อ	ก
ABSTRACT	ข
กิตติกรรมประกาศ	ค
สารบัญ	ง
สารบัญตาราง	ช
สารบัญภาพ	ณ
บทที่ 1 ที่มาและความสำคัญ.....	1
1.1 ที่มาและความสำคัญของโครงงาน.....	1
1.3 ขอบเขตในการจัดทำโครงงาน	2
1.4 ระยะเวลาในการทำโครงงาน.....	2
บทที่ 2 ทฤษฎีและอุปกรณ์	4
2.1 SPEECH RECOGNITION.....	4
2.1.1 Speech Signal Characteristics	4
2.1.2 Analog to Digital Conversion	4
2.1.3 Framing and Windowing	5
2.1.4 Fast Fourier Transform.....	6
2.1.5 Power Spectrum	7
2.1.6 Mel Filter Bank	7
2.1.7 Logarithm	8
2.1.8 Discrete Cosine Transform (DCT)	8
2.1.9 Acoustic Model	10
2.1.10 Language Model	10
2.1.11 Decoding.....	10
2.2 CAN (CONTROLLER AREA NETWORK).....	11
2.2.1 CAN (Controller Area Network)	11
2.2.2 Physical Layer of Can.....	11
2.2.3 CAN Data Frame.....	12
2.2.4 การคำนวณอัตราการส่งข้อมูล (Bit Rate)	13
2.3 SPI (SERIAL PERIPHERAL INTERFACE)	13
2.3.1 SPI (Serial Peripheral Interface).....	13

สารบัญ

	หน้า
2.3.2 Physical Layer of SPI	13
2.4 MCP2515 (MCP2515 STAND-ALONE CAN CONTROLLER).....	14
2.4.1 MCP2515.....	14
2.4.2 โครงสร้างภายใน MCP2515	14
2.5 TJA1050 (HIGH-SPEED CAN TRANSCEIVER)	15
2.5.1 TJA1050.....	15
2.5.2 โครงสร้างภายใน TJA1050	15
2.6 SN65HVD230DR (3.3V CAN TRANSCEIVER)	16
2.6.1 SN65HVD230DR	16
2.6.2 โครงสร้างภายในของ SN65HVD230DR	16
2.7 ESP32 (ESP32 DEVELOPMENT BOARD).....	17
2.7.1 ESP32 (ESP32 Development Board)	17
2.7.2 โครงสร้างและส่วนประกอบ	18
2.7.3 ชุดควบคุมการสื่อสาร TWAI (Two-Wire Automotive Interface)	18
2.8 RASPBERRY PI 4 MODEL B.....	18
2.8.1 Raspberry Pi 4 Model B	18
2.8.2 พอร์ตอินพุต/เอาต์พุตเนกประสงค์ (GPIO) และการเชื่อมต่อระบบแคน (CAN Bus Integration).....	19
2.9 TFT LCD WITH ILI9341.....	20
2.9.1 TFT LCD	20
2.9.2 ILI9341	20
2.10 FREERTOS (FREE REAL-TIME OPERATING SYSTEM)	20
2.10.1 FreeRTOS (Free Real-Time Operating System)	20
2.10.2 FreeRtos on ESP32	21
2.10.3 องค์ประกอบหลักในการจัดการทรัพยากร (Key Core Components)	21
บทที่ 3 หลักการทำงานและการออกแบบ	23
3.1 SYSTEM ARCHITECTURE.....	23
3.2 WAKE WORD DETECTION.....	23
3.3 SPEECH RECOGNITION.....	23
3.4 INTENT PARSING.....	24

สารบัญ

	หน้า
3.4.1 Hybrid Intent Parsing Architecture.....	25
3.4.2 Local Intent Parser	25
3.4.3 AI-based Intent Parser.....	26
3.4.4 Intent Output Structure.....	26
3.5 INTENT-TO-CAN MAPPING.....	26
3.5.1 CAN Data Field Structure	27
3.5.2 Rolling Counter	28
3.5.3 CRC8 Error Detection	28
3.6 CAN MESSAGE TRANSMISSION	31
3.6.1 CAN Message Transmission.....	31
3.6.2 CAN Message Construction	32
3.7 BLOCK DIAGRAM HARDWARE (SCHEMATIC)	32
3.7.1 โหนดประมวลผลกลาง (Master Node: Raspberry Pi 4 Model B).....	32
3.7.2 CAN Bus Physical Layer	33
3.7.3 Local Node: ESP32 Dev Module และหน้าจอ TFT	33
3.8 โครงสร้างสถาปัตยกรรมซอฟต์แวร์ (SOFTWARE ARCHITECTURE).....	33
3.9 ระบบการรับข้อมูลจาก CAN BUS (TWAI CONTROLLER INTEGRATION).....	34
3.9.1 Hardware Configuration	34
3.9.2 Message Parsing & Protocol.....	35
3.10 ระบบแสดงผล (TFT DISPLAY & UI MANAGEMENT)	35
3.10.1 การอัปเดตเฉพาะจุด (Delta Rendering).....	35
3.10.2 การวาดกราฟิกและแอนิเมชัน	35
3.3.3 ควบคุมฮาร์ดแวร์ระดับรีจิสเตอร์ (Register-Level Hardware Control)	36
3.4 PCB LAYOUT.....	36
3.5 Flowchart software การทำงานของระบบ.....	37
3.5.1 ภาคฝั่งการประมวลผลอินพุต และ การส่งข้อมูล.....	37
3.5.2 ภาคฝั่งการแสดงผลเอาต์พุต และ การรับข้อมูล.....	38
3.6 Block diagram.....	40
บทที่ 4 ผลการทดลอง.....	41
4.1 ทดลองหน้าจอการเปิดระบบ	41

สารบัญ

	หน้า
4.2 ทดลองหน้าจอแสดงผลของรถยนต์.....	41
4.3 ทดลองคำสั่งเปิดแอร์.....	42
4.4 ทดลองคำสั่งปรับแอร์เป็นอุณหภูมิ 20 องศา.....	42
4.5 ทดลองคำสั่งปิดแอร์.....	43
4.6 ทดลองคำสั่งเปิดกระจกทั้งหมด	43
4.6 ทดลองคำสั่งเปิดกระจกทั้งหมด	43
4.7 ทดลองคำสั่งเปิดกระจกฝั่งคนขับครึ่งบาน	44
4.8 ทดลองคำสั่งปิดกระจกทั้งหมด.....	45
4.9 ทดลองคำสั่งเปิดไฟในรถ.....	45
4.10 ทดลองคำสั่งปิดไฟในรถ.....	46
4.11 ทดสอบหาจุดบกพร่อง และ การหน่วงของระบบ.....	46
บทที่ 5 สรุปผลการทดลอง	47
5.1 สรุปภาพรวมผลการทำงานของระบบ	47
5.2 ปัญหาและอุปสรรคที่พบ.....	47
อ้างอิง	46

สารบัญตาราง

ตารางที่	หน้า
ตารางที่ 1 กำหนดการณ์ดำเนินการ	2
ตารางที่ 2 Intent ที่รองรับในระบบ.....	24
ตารางที่ 3 CAN ID สำหรับอุปกรณ์หลัก	27
ตารางที่ 4 HVAC Command Frame	27
ตารางที่ 5 Window Command Frame	27
ตารางที่ 6 Light Command Frame	28
ตารางที่ 7 คำอธิบาย.....	29
ตารางที่ 8 คำอธิบาย.....	30
ตารางที่ 9 คำอธิบาย.....	30
ตารางที่ 10 คำอธิบาย.....	31
ตารางที่ 11 สรุปการทดสอบหาจุดบกพร่อง และ การหน่วงของระบบ.....	45

สารบัญภาพ

รูปที่	หน้า
รูปที่ 1 Audio Framing.....	5
รูปที่ 2 Hamming Window	6
รูปที่ 3 Mel Filter Bank	8
รูปที่ 4 Flowchart.....	10
รูปที่ 5 CAN Bus (Controller Area Network).....	11
รูปที่ 6 CAN Bus (Physical Layer) Differential Signal.....	11
รูปที่ 7 CAN Data Frame.....	12
รูปที่ 8 เชื่อมต่อสื่อสารแบบ SPI (Serial Peripheral Interface)	13
รูปที่ 9 MCP2515.....	14
รูปที่ 10 TJA1050	15
รูปที่ 11 SN65HVD230DR	17
รูปที่ 12 ESP32 (ESP32 Development Board).....	17
รูปที่ 13 Raspberry Pi 4 Model B.....	19
รูปที่ 14 GPIO ของ Raspberry Pi 4 Model B	19
รูปที่ 15 TFT LCD	20
รูปที่ 16 การทำงานของ RTOS (Real-Time Operating System).....	21
รูปที่ 17 Block Diagram Hardware	32
รูปที่ 18 การทำงานของ FreeRTOS บนบอร์ด ESP32 ในรูปแบบของ Execution Timeline	34
รูปที่ 19 กราฟิก.....	35
รูปที่ 20 แผ่นผัง PCB.....	36
รูปที่ 21 FlowChart software ภาคฝั่งการประมวลผลอินพุต และ การส่งข้อมูล.....	37
รูปที่ 22 FlowChart software ภาคฝั่งการแสดงผลเอาต์พุต และ การรับข้อมูล.....	38
รูปที่ 23 Block Diagram Voice-Controlled In-Vehicle System.....	39
รูปที่ 24 ทดลองหน้าจอการเปิดระบบ	41
รูปที่ 25 ทดลองหน้าจอแสดงผลของรถยนต์	41
รูปที่ 26 หน้าจอทดลองคำสั่งเปิดแอร์ และสัญญาณทดลองคำสั่งเปิดแอร์.....	42
รูปที่ 27 หน้าจอทดลองคำสั่งปรับแอร์เป็นอุณหภูมิ 20 องศา และสัญญาณทดลองคำสั่งปรับแอร์เป็นอุณหภูมิ 20 องศา.....	42
รูปที่ 28 หน้าจอทดลองคำสั่งปิดแอร์ และสัญญาณทดลองคำสั่งปิดแอร์	43
รูปที่ 29 หน้าจอทดลองคำสั่งเปิดกระจกทั้งหมด และสัญญาณทดลองคำสั่งเปิดกระจกทั้งหมด	43

สารบัญภาพ

รูปที่	หน้า
รูปที่ 30 หน้าจอทดลองคำสั่งเปิดกระจกทั้งหมด และสัญญาณทดลองคำสั่งเปิดกระจกทั้งหมด	44
รูปที่ 31 หน้าจอทดลองคำสั่งเปิดกระจกฝั่งคนขับครึ่งบาน และสัญญาณทดลองคำสั่งเปิดกระจกฝั่งคนขับครึ่งบาน.....	44
รูปที่ 32 หน้าจอทดลองคำสั่งปิดกระจกทั้งหมด และสัญญาณทดลองคำสั่งปิดกระจกทั้งหมด	45
รูปที่ 33 หน้าจอทดลองคำสั่งเปิดไฟในรถ และสัญญาณทดลองคำสั่งเปิดไฟในรถ	45
รูปที่ 34 หน้าจอทดลองคำสั่งปิดไฟในรถ และสัญญาณทดลองคำสั่งปิดไฟในรถ.....	46

บทที่ 1

ที่มาและความสำคัญ

1.1 ที่มาและความสำคัญของโครงการ

ในปัจจุบันเทคโนโลยีระบบอิเล็กทรอนิกส์ในยานยนต์มีการพัฒนาอย่างต่อเนื่อง โดยรถยนต์สมัยใหม่มีการใช้ระบบเครือข่ายสื่อสารภายในรถยนต์ เช่น Controller Area Network (CAN Bus) เพื่อเชื่อมต่อและควบคุมอุปกรณ์อิเล็กทรอนิกส์ต่าง ๆ ภายในรถยนต์ เช่น ระบบปรับอากาศ ระบบกระจกไฟฟ้า และระบบไฟภายในรถยนต์ การใช้เครือข่าย CAN ช่วยให้อุปกรณ์ต่าง ๆ สามารถสื่อสารและแลกเปลี่ยนข้อมูลกันได้อย่างมีประสิทธิภาพ รวมทั้งช่วยลดความซับซ้อนของการเดินสายไฟภายในรถยนต์

ในขณะเดียวกัน เทคโนโลยี การรู้จำเสียงพูด (Speech Recognition) ได้ถูกนำมาใช้ในระบบอัจฉริยะหลายประเภท เพื่อช่วยให้ผู้ใช้สามารถควบคุมอุปกรณ์ต่าง ๆ ได้สะดวกมากยิ่งขึ้นผ่านคำสั่งเสียง ดังนั้นโครงการนี้จึงมีวัตถุประสงค์เพื่อพัฒนาระบบควบคุมอุปกรณ์ภายในรถยนต์ผ่านคำสั่งเสียง โดยใช้ Raspberry Pi เป็นหน่วยประมวลผลในการรับและประมวลผลคำสั่งเสียง จากนั้นแปลงคำสั่งดังกล่าวให้เป็น CAN message เพื่อส่งผ่านเครือข่าย CAN Bus ไปควบคุมอุปกรณ์ต่าง ๆ ภายในระบบ เช่น ระบบปรับอากาศ กระจกไฟฟ้า และไฟภายในรถยนต์ ซึ่งช่วยเพิ่มความสะดวกในการใช้งานและเป็นแนวทางในการประยุกต์ใช้เทคโนโลยีการรู้จำเสียงร่วมกับระบบสื่อสารในยานยนต์

1.2 วัตถุประสงค์ของโครงการ

1.2.1 เพื่อพัฒนาระบบควบคุมอุปกรณ์ภายในรถยนต์ผ่านคำสั่งเสียงโดยใช้ Raspberry Pi เป็นหน่วยประมวลผลหลัก

1.2.2 เพื่อศึกษาการประยุกต์ใช้เทคโนโลยี Speech Recognition ในการรับและแปลงคำสั่งเสียงของผู้ใช้ให้เป็นข้อความคำสั่ง

1.2.3 เพื่อออกแบบและพัฒนาระบบแปลงคำสั่งเสียงให้อยู่ในรูปแบบ CAN message สำหรับใช้สื่อสารผ่านเครือข่าย CAN Bus

1.2.4 เพื่อทดสอบการควบคุมอุปกรณ์ภายในระบบ เช่น ระบบปรับอากาศ กระจกไฟฟ้า และไฟภายในรถยนต์ ผ่านเครือข่าย CAN Bus

1.2.5 เพื่อออกแบบและพัฒนาระบบแสดงผลสถานะอุปกรณ์ภายในรถยนต์แบบเรียลไทม์ (Real-time) บนหน้าจอ TFT

1.2.6 เพื่อประยุกต์ใช้ระบบปฏิบัติการเวลาจริง (FreeRTOS) ในการจัดการสถาปัตยกรรมซอฟต์แวร์บนไมโครคอนโทรลเลอร์ ให้สามารถประมวลผลการรับข้อมูลจาก CAN Bus และตอบสนองหน้าจอแสดงผลคู่ขนานกันได้อย่างมีประสิทธิภาพ

1.3 ขอบเขตในการจัดทำโครงการงาน

1.3.1 โครงการนี้พัฒนาระบบควบคุมอุปกรณ์ภายในรถยนต์ผ่านคำสั่งเสียง โดยใช้ Raspberry Pi เป็นหน่วยประมวลผลหลักของระบบ

1.3.2 ระบบรับคำสั่งเสียงจากผู้ใช้ผ่านไมโครโฟน และทำการแปลงสัญญาณเสียงเป็นข้อความด้วยเทคโนโลยี Speech Recognition

1.3.3 ข้อความคำสั่งเสียงจะถูกนำไปวิเคราะห์เพื่อระบุประเภทของคำสั่ง (Intent) และแปลงให้อยู่ในรูปแบบคำสั่งสำหรับควบคุมอุปกรณ์

1.3.4 ระบบทำการแปลงคำสั่งให้อยู่ในรูปแบบ CAN message และส่งข้อมูลผ่านเครือข่าย CAN Bus เพื่อควบคุมอุปกรณ์ในระบบ

1.3.5 การควบคุมอุปกรณ์ในโครงการนี้ครอบคลุมการทำงานของระบบ เครื่องปรับอากาศ กระบอกไฟฟ้า และไฟภายในรถยนต์

1.3.6 การทดลองและทดสอบระบบจะดำเนินการในสภาพแวดล้อมของชุดทดลองที่จำลองการทำงานของระบบ CAN ภายในรถยนต์

1.4 ระยะเวลาในการทำโครงการงาน

ตารางที่ 1 กำหนดการณั้ดำเนินการ

รายละเอียด	เดือน				ผู้รับผิดชอบ
	ธ.ค. 2568	ม.ค. 2568	ก.พ. 2569	มี.ค. 2569	
ศึกษาการแปลงเสียงเป็นข้อความด้วย Speech Recognition					
ศึกษาการสื่อสารข้อมูลด้วย CAN Protocol					
ทดลองใช้ library speech recognition					
ทดลองการส่งข้อมูลผ่าน CAN Bus					

รายละเอียด	เดือน				ผู้รับผิดชอบ
	จ.ค. 2568	ม.ค. 2568	ก.พ. 2569	มี.ค. 2569	
ออกแบบ schematic ของระบบ					
ทดสอบการส่ง CAN ID และ CAN Data จาก Raspberry Pi					
ทดสอบ GUI สำหรับจอ TFT					
ทดสอบระบบประมวลผลคำสั่งเสียง					
ออกแบบ Layout บอร์ดฝั่งรับและฝั่งส่ง					
ลงอุปกรณ์บนบอร์ด					
ทดสอบระบบ CAN Bus และแสดงผลลัพท์					
จัดทำรูปเล่มรายงานโครงการ					
นำเสนอโครงการ					

บทที่ 2

ทฤษฎีและอุปกรณ์

2.1 Speech Recognition

Speech Recognition หรือ Speech-to-Text (STT) เป็นเทคโนโลยีที่ใช้ในการแปลงสัญญาณเสียงพูดของมนุษย์ให้เป็นข้อความในรูปแบบดิจิทัล โดยอาศัยการประมวลผลสัญญาณดิจิทัล (Digital Signal Processing: DSP) ร่วมกับเทคนิคด้านปัญญาประดิษฐ์และการเรียนรู้ของเครื่องเพื่อวิเคราะห์ลักษณะของเสียงและระบุคำที่ผู้พูดกล่าวออกมา ปัจจุบันเทคโนโลยี Speech Recognition ถูกนำไปใช้อย่างแพร่หลายในหลายระบบ เช่น ระบบผู้ช่วยอัจฉริยะ การสั่งงานด้วยเสียง และระบบควบคุมอุปกรณ์อัตโนมัติ โดยระบบจะรับสัญญาณเสียงจากไมโครโฟน แปลงเป็นสัญญาณดิจิทัล และวิเคราะห์รูปแบบของเสียงเพื่อแปลงเป็นข้อความที่สอดคล้องกับคำพูดของผู้ใช้

2.1.1 Speech Signal Characteristics

สัญญาณเสียงพูด (speech signal) เป็นสัญญาณที่เกิดจากการสั่นของสายเสียง และการเปลี่ยนแปลงของทางเดินเสียง (vocal tract) ซึ่งทำให้เกิดคลื่นเสียงที่มีความถี่และแอมพลิจูดแตกต่างกัน เสียงพูดของมนุษย์มีลักษณะเป็น Non-stationary Signal เนื่องจากคุณสมบัติของสัญญาณ เช่น ความถี่และพลังงานของสัญญาณมีการเปลี่ยนแปลงตามเวลา อย่างไรก็ตาม ในช่วงเวลาสัญญาณเสียงประมาณ 20–30 มิลลิวินาทีสามารถประมาณได้ว่าสัญญาณมีลักษณะ Quasi-stationary ซึ่งทำให้สามารถนำเทคนิคการประมวลผลสัญญาณ เช่น การวิเคราะห์ในโดเมนความถี่ มาใช้กับสัญญาณเสียงพูดได้โดยทั่วไปความถี่ของเสียงพูดมนุษย์จะอยู่ในช่วงประมาณ 300 Hz ถึง 3400 Hz ซึ่งเป็นช่วงความถี่ที่สำคัญสำหรับการสื่อสารและการรู้จำเสียงพูด

2.1.2 Analog to Digital Conversion

เนื่องจากสัญญาณเสียงพูดเป็นสัญญาณแบบแอนะล็อก (analog signal) แต่ระบบคอมพิวเตอร์สามารถประมวลผลข้อมูลในรูปแบบ ดิจิทัล (digital signal) ดังนั้นสัญญาณเสียงจึงต้องถูกแปลงเป็นสัญญาณดิจิทัลผ่านกระบวนการ Analog-to-Digital Conversion (ADC) ขั้นตอนหลักของการแปลงสัญญาณประกอบด้วย Sampling ซึ่งเป็นการเก็บค่าของสัญญาณเสียงในช่วงเวลาที่กำหนด โดยสามารถเขียนได้เป็น

$$X[N] = X(nT_s) \quad (1)$$

เมื่อ

$X(T)$ คือสัญญาณเสียงแบบต่อเนื่อง

$X[N]$ คือสัญญาณเสียงแบบดิจิทัล

T_s คือช่วงเวลาในการ Sampling

ตาม Nyquist Theorem อัตราการ Sampling ต้องมีค่ามากกว่าสองเท่าของความถี่สูงสุดของสัญญาณ เพื่อป้องกันการสูญเสียข้อมูลของสัญญาณ ในระบบ Speech Recognition โดยทั่วไปจะใช้อัตราการ Sampling ประมาณ 16 kHz ซึ่งเพียงพอสำหรับการประมวลผลเสียงพูดของมนุษย์ ก่อนที่จะนำสัญญาณไปสู่ขั้นตอนการประมวลผลสัญญาณในขั้นตอนถัดไป เช่น Framing และ Feature Extraction

$$f_s \geq 2f_{max} \quad (2)$$

2.1.3 Framing and Windowing

เนื่องจากสัญญาณเสียงพูดเป็น Non-stationary signal ซึ่งมีคุณสมบัติของสัญญาณที่เปลี่ยนแปลงตามเวลา ดังนั้นเพื่อให้สามารถวิเคราะห์สัญญาณได้ง่ายขึ้น จึงต้องทำการแบ่งสัญญาณเสียงออกเป็นช่วงเวลาสั้น ๆ ซึ่งเรียกว่าเฟรมโดยทั่วไปความยาวของเฟรมจะอยู่ประมาณ 20–25 มิลลิวินาที และมีการซ้อนทับกันระหว่างเฟรมประมาณ 10 มิลลิวินาที เพื่อให้ข้อมูลของสัญญาณมีความต่อเนื่องและไม่สูญเสียข้อมูลสำคัญของเสียงพูดหากกำหนดให้ $X[N]$ คือสัญญาณเสียงแบบดิจิทัล สัญญาณในแต่ละเฟรมสามารถเขียนได้ดังสมการ (3) การแบ่งสัญญาณเป็นเฟรมช่วยให้สามารถวิเคราะห์สัญญาณในช่วงเวลาสั้น ๆ ที่ถือว่าสัญญาณมีลักษณะค่อนข้างคงที่

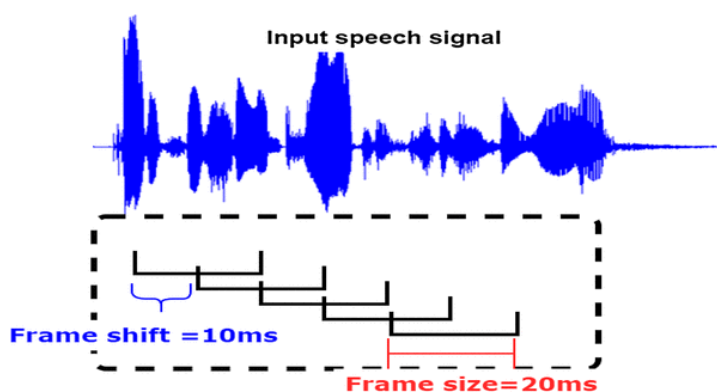
$$X_m[N] = x[n + mR] \quad (3)$$

เมื่อ

$X_m[N]$ คือสัญญาณเสียงแบบต่อเนื่อง

R คือจำนวน sample ที่เลื่อนของแต่ละ Frame

n คือดัชนีของ sample



รูปที่ 1 Audio Framing

หลังจากแบ่งสัญญาณเป็นเฟรมแล้วจะทำการใช้ Window Function เพื่อปรับรูปแบบของสัญญาณในแต่ละเฟรมก่อนนำไปวิเคราะห์ในโดเมนความถี่เนื่องจากการตัดสัญญาณโดยตรงอาจทำให้เกิดความไม่ต่อเนื่องของสัญญาณที่ขอบเฟรม ซึ่งอาจส่งผลให้เกิดความผิดเพี้ยนของสเปกตรัม การทำ Windowing คือผลคูณระหว่างสัญญาณในเฟรมกับ Window Function ดังสมการ (4)

$$x_w[N] = x[n]w[n] \quad (4)$$

เมื่อ

$x_w[N]$ คือสัญญาณหลังจากการทำ windowing

$w[N]$ คือ window function

$x[N]$ คือสัญญาณเสียงในเฟรม

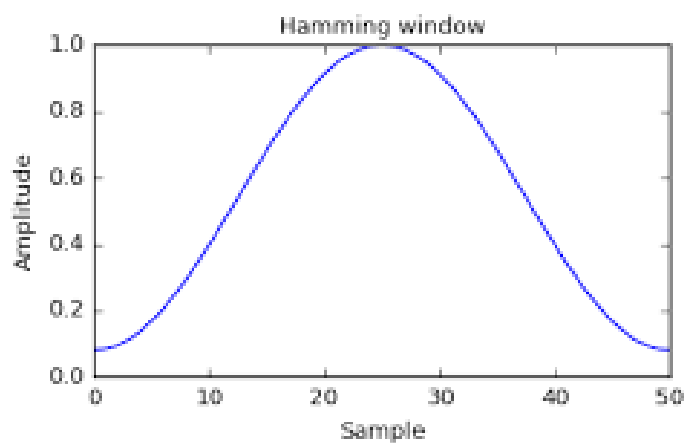
Window Function ที่นิยมใช้ในระบบ Speech Recognition คือ Hamming Window แสดงในสมการ (5)

$$w[n] = 0.54 - 0.46 \cos\left(\frac{2\pi n}{N-1}\right) \quad (5)$$

เมื่อ

N คือจำนวน sample ในแต่ละเฟรม

การทำ Windowing ช่วยลดปัญหา Spectral Leakage และทำให้การวิเคราะห์สัญญาณในขั้นตอนถัดไป เช่น Fourier Transform มีความแม่นยำมากขึ้น



รูปที่ 2 Hamming Window

2.1.4 Fast Fourier Transform

Fourier Transform เป็นเทคนิคที่ใช้ในการแปลงสัญญาณจาก Time Domain ไปเป็น Frequency Domain เพื่อวิเคราะห์องค์ประกอบความถี่ของสัญญาณเสียงพูด ในทางปฏิบัตินิยมใช้ Fast Fourier Transform (FFT) เนื่องจากสามารถคำนวณได้รวดเร็ว สมการของ Discrete Fourier Transform สามารถเขียนได้ดังสมการ (6)

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N} \quad (6)$$

เมื่อ

$X[n]$ คือสัญญาณในโดเมนเวลา

$X[k]$ คือสัญญาณในโดเมนความถี่

N คือจำนวน sample ในแต่ละเฟรม

2.1.5 Power Spectrum

หลังจากทำการแปลงสัญญาณเสียงในแต่ละเฟรมด้วย Fast Fourier Transform (FFT) แล้ว จะได้สเปกตรัมของสัญญาณในโดเมนความถี่ จากนั้นจะทำการคำนวณ Power Spectrum เพื่อหาพลังงานของสัญญาณในแต่ละความถี่ สมการของ Power Spectrum สามารถเขียนได้ดังสมการ (7)

$$P[k] = (x[k])^2 \quad (7)$$

เมื่อ

$X[k]$ คือผลลัพธ์จากการทำ Fourier transform

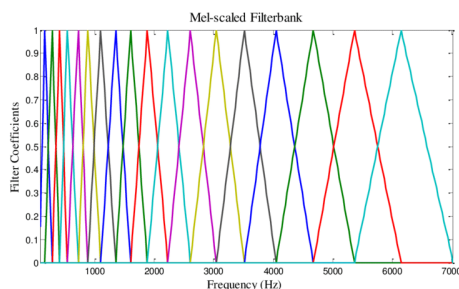
$P[k]$ คือพลังงานของสัญญาณในความถี่ k

Power Spectrum ช่วยแสดงการกระจายของพลังงานในแต่ละช่วงความถี่ของสัญญาณเสียง ซึ่งเป็นข้อมูลสำคัญในการวิเคราะห์ลักษณะของเสียงพูด

2.1.6 Mel Filter Bank

ขั้นตอนถัดไปคือการนำ Power Spectrum ไปผ่าน Mel Filter Bank ซึ่งเป็นชุดของตัวกรองความถี่แบบสามเหลี่ยม (Triangular Filters) ที่กระจายอยู่ตาม Mel Scale เพื่อจำลองการรับรู้ความถี่ของมนุษย์ มนุษย์มีการรับรู้ความถี่แบบไม่เป็นเชิงเส้น กล่าวคือสามารถแยกความแตกต่างของความถี่ต่ำได้ดีกว่าความถี่สูง ดังนั้นจึงมีการแปลงสเกลความถี่จากหน่วย Hertz ไปเป็น Mel scale ตามสมการ (8)

$$mel(f) = 2595 \log_{10} \left(1 + \frac{f}{700} \right) \quad (8)$$



รูปที่ 3 Mel Filter Bank

ตัวกรองใน Mel Filter Bank จะรวมพลังงานของสัญญาณในแต่ละช่วงความถี่ให้ได้ค่าพลังงานของแต่ละตัวกรอง ซึ่งช่วยลดจำนวนข้อมูลของสเปกตรัมและเน้นลักษณะที่สำคัญของเสียงพูด

2.1.7 Logarithm

หลังจากผ่าน Mel Filter Bank แล้ว จะทำการคำนวณค่า Logarithm ของพลังงานสัญญาณ ในแต่ละตัวกรอง

$$S_i = \log(E_i) \quad (9)$$

เมื่อ

S_i คือพลังงานที่ได้จากตัวกรอง mel filter

E_i คือพลังงานหลังการทำ log

การใช้ Logarithm ช่วยปรับสเกลของพลังงานให้เหมาะสมกับการประมวลผล อีกทั้งยังช่วยจำลองลักษณะการรับรู้ความดังของเสียงของมนุษย์ ซึ่งมีลักษณะเป็นแบบลอการิทึม

2.1.8 Discrete Cosine Transform (DCT)

ขั้นตอนสุดท้ายคือการใช้ Discrete Cosine Transform (DCT) เพื่อแปลงค่าพลังงานที่ได้จาก Mel filter bank ให้อยู่ในรูปของ Cepstral Coefficients สมการ DCT แสดงใน (10)

$$c_n = \sum_{k=1}^K S_k \cdot \cos \left[\frac{\pi n}{k} \cdot (k - 0.5) \right] \quad (10)$$

เมื่อ

C_n คือค่า cepstral coefficient ลำดับที่ n

S_k คือค่าพลังงานหลังการทำ log

K คือจำนวน mel filter

ผลลัพธ์สุดท้ายของขั้นตอนนี้คือ MFCC Feature Vector ซึ่งเป็นชุดของตัวเลขที่ใช้แทนลักษณะของเสียงพูดในแต่ละเฟรม โดยทั่วไปจะใช้ประมาณ 12–13 coefficients ต่อหนึ่งเฟรม

MFCC Feature Vector ที่ได้จะถูกนำไปใช้เป็น ข้อมูลนำเข้าสำหรับโมเดลรู้จำเสียงพูด (Acoustic Model) ในขั้นตอนถัดไปของระบบ Speech Recognition นอกจากค่า MFCC coefficients ที่ได้จากแต่ละเฟรมแล้ว ระบบ Speech Recognition มักเพิ่ม Dynamic Features เพื่อสะท้อนการเปลี่ยนแปลงของสัญญาณเสียงตามเวลา เนื่องจากเสียงพูดมีลักษณะเป็นสัญญาณที่เปลี่ยนแปลงอย่างต่อเนื่อง การใช้เฉพาะ MFCC ของแต่ละเฟรมอาจไม่สามารถแสดงลักษณะการเปลี่ยนแปลงของเสียงได้อย่างครบถ้วน Dynamic Features ที่นิยมใช้ประกอบด้วย

1. Delta coefficients (Δ)
2. Delta-Delta coefficients (Δ^2)

Delta coefficients เป็นค่าที่แสดงอัตราการเปลี่ยนแปลงของ MFCC ตามเวลา ซึ่งสามารถคำนวณได้จากความแตกต่างระหว่างเฟรมใกล้เคียงดังสมการ (11)

$$\Delta = \frac{\sum_{n=1}^N n(c_{t+n} - c_{t-n})}{2 \cdot \sum_{n=1}^N n^2} \quad (11)$$

เมื่อ

c_t คือ ค่า MFCC ของเฟรมที่ t

N คือจำนวนเฟรมที่ใช้ในการคำนวณ

Δ คือ delta coefficient

Delta-Delta coefficients หรือ Acceleration coefficients เป็นค่าที่แสดงอัตราการเปลี่ยนแปลงของ Delta coefficients ซึ่งสามารถคำนวณได้ในลักษณะเดียวกันด้วยสมการ (12)

$$\Delta^2 = \frac{\sum_{n=1}^N n(\Delta_{t+n} - \Delta_{t-n})}{2 \cdot \sum_{n=1}^N n^2} \quad (12)$$

ดังนั้น feature vector ที่ใช้ในระบบ Speech Recognition มักประกอบด้วย

1. MFCC coefficients
2. Delta-Delta coefficients (Δ^2)

ตัวอย่างเช่น หากใช้ MFCC จำนวน 13 ค่า จะได้ feature vector ทั้งหมด

$$13 + 13 + 13 = 39 \quad (13)$$

ซึ่งช่วยให้โมเดลสามารถเรียนรู้ ลักษณะของเสียงและการเปลี่ยนแปลงของเสียงตามเวลา ได้ดีขึ้น ส่งผลให้ความแม่นยำของระบบ Speech Recognition เพิ่มขึ้นอย่างมีนัยสำคัญ

2.1.9 Acoustic Model

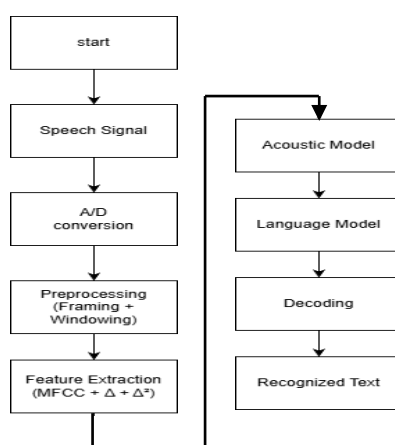
Acoustic Model เป็นส่วนสำคัญของระบบ Speech Recognition ที่ทำหน้าที่เชื่อมโยง คุณลักษณะของเสียงพูด (Feature vectors) เช่น MFCC กับ หน่วยเสียง (Phoneme) ของภาษา โดยโมเดลจะทำการเรียนรู้รูปแบบของเสียงจากข้อมูลฝึก (training data) เพื่อคำนวณความน่าจะเป็นของหน่วยเสียงที่สอดคล้องกับสัญญาณเสียงที่รับเข้ามา ซึ่งจะถูกส่งต่อไปยังขั้นตอน Decoding และ Language Model เพื่อสร้างคำหรือประโยคที่ผู้พูดกล่าว

2.1.10 Language Model

Language Model เป็นส่วนสำคัญของระบบ Speech Recognition ที่ทำหน้าที่กำหนดความน่าจะเป็นของลำดับคำในภาษา เพื่อช่วยให้ระบบสามารถเลือกคำหรือประโยคที่มีความสมเหตุสมผลมากที่สุดจากผลลัพธ์ที่ได้จาก Acoustic Model แม้ว่า Acoustic Model จะสามารถทำนายหน่วยเสียงหรือ phoneme จากสัญญาณเสียงได้ แต่ในหลายกรณีเสียงบางคำอาจมีลักษณะคล้ายกัน ทำให้เกิดความคลุมเครือในการรู้จำคำพูด ตัวอย่างเช่นคำที่มีเสียงใกล้เคียงกันอาจถูกตีความได้หลายแบบ ดังนั้น Language Model จึงถูกใช้เพื่อประเมินว่าลำดับคำใดมีความเป็นไปได้มากที่สุดตามหลักไวยากรณ์ของภาษา

2.1.11 Decoding

Decoding เป็นขั้นตอนสุดท้ายของระบบ Speech Recognition ที่ทำหน้าที่รวมผลลัพธ์จาก Acoustic Model และ Language Model เพื่อหาลำดับคำที่มีความน่าจะเป็นมากที่สุดจากสัญญาณเสียงที่รับเข้ามา โดยขั้นตอนนี้จะทำการค้นหา (search) ลำดับของคำหรือ phoneme ที่สอดคล้องกับสัญญาณเสียงมากที่สุด

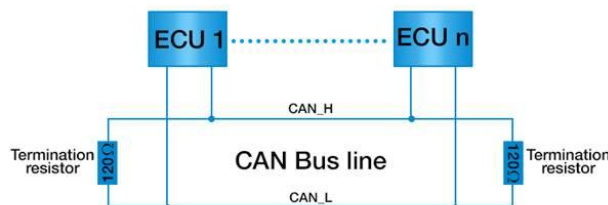


รูปที่ 4 Flowchart

2.2 Can (Controller Area Network)

2.2.1 CAN (Controller Area Network)

โพรโตคอลมาตรฐานการสื่อสารข้อมูลแบบอนุกรม (Serial Communication) เพื่อใช้ในระบบเครือข่ายของยานยนต์ โดยมีจุดประสงค์หลักเพื่อลดปริมาณสายไฟและเพิ่มความน่าเชื่อถือในการส่งข้อมูลระหว่างอุปกรณ์อิเล็กทรอนิกส์ต่างๆ (ECUs) ภายในรถยนต์ ระบบ CAN มีลักษณะการเชื่อมต่อเครือข่ายแบบบัส (Bus Topology) และทำงานแบบ Multi-master ซึ่งหมายความว่าทุกโหนด (Node) ในเครือข่ายสามารถเริ่มต้นการส่งข้อมูลได้โดยไม่ต้องรอคำสั่งจากโหนดศูนย์กลาง นอกจากนี้ ที่ปลายสายของเครือข่ายจะต้องมีการต่อตัวต้านทานปิดท้าย (Terminating Resistor) ขนาด 120 โอห์ม เพื่อป้องกันการสะท้อนกลับของสัญญาณ (Signal Reflection)

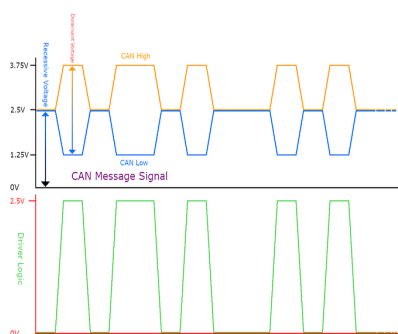


รูปที่ 5 CAN Bus (Controller Area Network)

2.2.2 Physical Layer of Can

การส่งสัญญาณของ CAN จะใช้สายคู่ตีเกลียว (Twisted Pair) จำนวน 2 เส้น ได้แก่ CAN_H (CAN High) และ CAN_L (CAN Low) โดยใช้หลักการส่งสัญญาณแบบความแตกต่างของแรงดันไฟฟ้า (Differential Signaling)

1. สถานะ Recessive (ลอจิก 1): แรงดันไฟฟ้าของ CAN_H และ CAN_L จะมีค่าเท่ากัน (ประมาณ 2.5V) ทำให้ค่าความต่างศักย์เป็น 0V
2. สถานะ Dominant (ลอจิก 0): แรงดันไฟฟ้าของ CAN_H จะสูงขึ้น (ประมาณ 3.5V) และ CAN_L จะลดลง (ประมาณ 1.5V) ทำให้ค่าความต่างศักย์อยู่ที่ประมาณ 2V



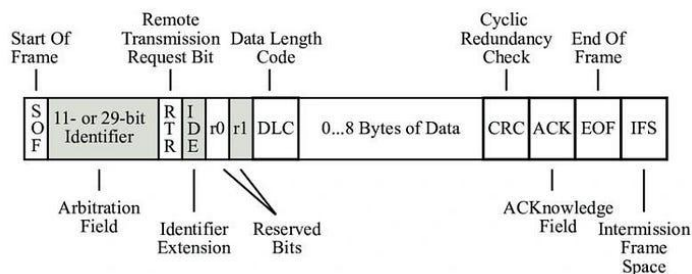
รูปที่ 6 CAN Bus (Physical Layer) Differential Signal

การใช้ระบบ Differential Signaling นี้ทำให้โปรโตคอล CAN มีความทนทานต่อสัญญาณรบกวนทางแม่เหล็กไฟฟ้า (EMI) ได้สูงมาก เหมาะสำหรับสภาพแวดล้อมที่มีสัญญาณรบกวนสูงในอุตสาหกรรมและยานยนต์

2.2.3 CAN Data Frame

Data Frame ในมาตรฐาน CAN 2.0A (Standard CAN) จะมีขนาดของ Identifier จำนวน 11 บิต โครงสร้างพื้นฐานประกอบด้วยส่วนสำคัญ ดังนี้

1. SOF (Start of Frame): บิตเริ่มต้นการส่งข้อมูล (Dominant bit)
2. Identifier (ID): รหัสระบุข้อความและกำหนดความสำคัญ (Priority) ยิ่งค่า ID น้อย ยิ่งมีความสำคัญสูงและได้สิทธิ์ส่งข้อมูลก่อน (Arbitration)
3. Control Field: ระบุจำนวนไบต์ของข้อมูลที่กำลังส่ง (DLC - Data Length Code)
4. Data Field: ข้อมูลที่ต้องการส่ง ขนาดตั้งแต่ 0 ถึง 8 ไบต์
5. CRC Field: รหัสตรวจสอบความถูกต้องของข้อมูล
6. ACK Field: บิตตอบรับจากโหนดผู้รับ
7. EOF (End of Frame): บิตสิ้นสุดเฟรมข้อมูล



รูปที่ 7 CAN Data Frame

2.2.4 การคำนวณอัตราการส่งข้อมูล (Bit Rate)

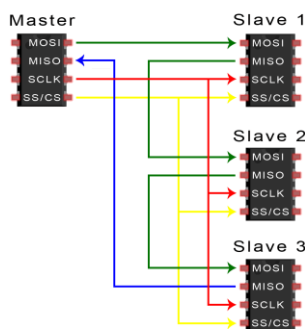
ความเร็วในการส่งข้อมูลของระบบ CAN (Bit Rate หรือ R) มีความสัมพันธ์ผกผันกับเวลาที่ใช้ในการส่งข้อมูล 1 บิต (Bit Time หรือ T_b) ซึ่งสามารถคำนวณได้จากสมการพื้นฐาน

$$T_b = \frac{1}{R} \quad (14)$$

2.3 SPI (Serial Peripheral Interface)

2.3.1 SPI (Serial Peripheral Interface)

โปรโตคอล SPI (Serial Peripheral Interface) เป็นมาตรฐานการสื่อสารข้อมูลแบบอนุกรมความเร็วสูงที่ทำงานแบบซิงโครนัส (Synchronous Serial Communication) ถูกเพื่อใช้สำหรับการสื่อสารระยะสั้นระหว่างไมโครคอนโทรลเลอร์และอุปกรณ์ต่อพ่วง (Peripheral devices) บนแผงวงจรเดียวกัน เช่น เซนเซอร์, หน่วยความจำ (EEPROM/SD Card) หรือโมดูลแสดงผล การสื่อสารแบบ SPI มีโครงสร้างการทำงานแบบผู้คุมและผู้ตาม (Master-Slave Architecture) โดยสามารถมีการรับและส่งข้อมูลสวนทางกันได้ในเวลาเดียวกัน (Full-Duplex) ซึ่งอุปกรณ์ที่เป็น Master จะเป็นผู้สร้างสัญญาณนาฬิกา (Clock) เพื่อควบคุมจังหวะการรับส่งข้อมูลเสมอ



รูปที่ 8 เชื่อมต่อสื่อสารแบบ SPI (Serial Peripheral Interface)

2.3.2 Physical Layer of SPI

โครงสร้างพื้นฐานของการเชื่อมต่อแบบ SPI จะใช้สายสัญญาณ 4 เส้นหลัก (4-Wire) ในการสื่อสาร ได้แก่

1. SCK (Serial Clock): สายสัญญาณนาฬิกา สร้างโดยอุปกรณ์ Master เพื่อกำหนดจังหวะเวลาในการรับส่งข้อมูล
2. MOSI (Master Out Slave In): สายส่งข้อมูลจาก Master ไปยัง Slave (ในอุปกรณ์รุ่นใหม่บางรุ่นอาจใช้ชื่อ COPI: Controller Out Peripheral In)

3. MISO (Master in Slave Out): สายส่งข้อมูลจาก Slave กลับไปยัง Master ในอุปกรณ์รุ่นใหม่บางรุ่นอาจใช้ชื่อ CIPO: Controller in Peripheral Out)

4. SS (Slave Select) หรือ CS (Chip Select): สายเลือกอุปกรณ์รับข้อมูล (Active Low) Master จะดึงสัญญาณเส้นนี้ให้เป็นสถานะลอจิก 0 เพื่อระบุว่าต้องการสื่อสารกับ Slave ตัวใด

2.4 MCP2515 (MCP2515 Stand-Alone CAN Controller)

2.4.1 MCP2515

MCP2515 เป็นไอซีควบคุมการสื่อสารผ่านบัสแคน (Stand-Alone CAN Controller) มีหน้าที่หลักในการจัดการโปรโตคอล CAN โดยสมบูรณ์แบบตามมาตรฐาน ซึ่งรองรับความเร็วในการสื่อสาร (Bit Rate) สูงสุดถึง 1 Mbps (Megabits per second) ความโดดเด่นของไอซีตัวนี้คือ การออกแบบให้สื่อสารกับไมโครคอนโทรลเลอร์ (MCU) ผ่านโปรโตคอล SPI (Serial Peripheral Interface) ทำให้ไมโครคอนโทรลเลอร์ทั่วไปที่ไม่มีชุดควบคุม CAN ในตัว



รูปที่ 9 MCP2515

2.4.2 โครงสร้างภายใน MCP2515

เพื่อให้การรับส่งข้อมูลผ่าน CAN Bus มีประสิทธิภาพและไม่สร้างภาระให้กับการประมวลผลของไมโครคอนโทรลเลอร์หลัก MCP2515 จึงมีโครงสร้างภายในที่สำคัญ ดังนี้

1. กลไกจัดการโปรโตคอลแคน (CAN Protocol Engine): ทำหน้าที่ดูแลเรื่องการสร้างเฟรมข้อมูล (Framing), การจัดการสิทธิบนสายบัส (Arbitration), การตรวจสอบข้อผิดพลาด (Error Checking/CRC), และการจัดการสถานะข้อผิดพลาด (Error Handling) โดยอัตโนมัติ

2. หน่วยความจำบัฟเฟอร์ (SRAM Buffers)

2.1 TX Buffers (Transmit Buffers): บัฟเฟอร์สำหรับส่งข้อมูลจำนวน 3 ชุด ช่วยให้สามารถเตรียมข้อมูลรอส่งได้หลายคิว

2.2 RX Buffers (Receive Buffers): บัฟเฟอร์สำหรับรับข้อมูลจำนวน 2 ชุด เพื่อเก็บข้อมูลที่รับจากสายบัสก่อนที่ไมโครคอนโทรลเลอร์จะอ่านผ่าน SPI

3. ตัวกรองและตัวมาสก์ข้อมูล (Acceptance Filters and Masks): ประกอบด้วยมาสก์ (Mask) จำนวน 2 ชุด และตัวกรอง (Filter) จำนวน 6 ชุด ทำหน้าที่คัดกรองเฉพาะ ID ของ CAN Frame ที่ระบบต้องการใช้งานเท่านั้นให้อนุญาตผ่านเข้าสู่ RX Buffers ได้ ช่วยลดภาระของไมโครคอนโทรลเลอร์ในการประมวลผลข้อมูลที่ไม่เกี่ยวข้อง

2.5 TJA1050 (High-Speed CAN Transceiver)

2.5.1 TJA1050

ไอซีรับส่งสัญญาณแคนความเร็วสูง (High-Speed CAN Transceiver) ที่พัฒนาโดยบริษัท NXP Semiconductors (เดิมคือ Philips Semiconductors) ทำหน้าที่เป็นตัวกลางในการเชื่อมต่อระหว่างตัวควบคุมโปรโตคอลแคน (CAN Protocol Controller เช่น MCP2515 หรือ MCU ที่มี CAN ในตัว) กับสายสัญญาณทางกายภาพ (Physical CAN Bus) หน้าที่หลักของ TJA1050 คือการรับข้อมูลระดับลอจิก (Logic Level) จากตัวควบคุมแคน แล้วแปลงให้อยู่ในรูปแบบสัญญาณความต่างศักย์ไฟฟ้า (Differential Signal) เพื่อส่งออกไปยังเครือข่าย และในทางกลับกัน จะรับสัญญาณความต่างศักย์จากเครือข่ายมาแปลงกลับเป็นสัญญาณระดับลอจิกเพื่อให้ตัวควบคุมแคนนำไปประมวลผลต่อไป ไอซีรุ่นนี้รองรับความเร็วในการรับส่งข้อมูล (Baud Rate) ตั้งแต่ 60 kbps ไปจนถึงสูงสุด 1 Mbps



รูปที่ 10 TJA1050

2.5.2 โครงสร้างภายใน TJA1050

1. ภาควิทยาส่งสัญญาณ (Transmitter): ทำหน้าที่แปลงลอจิกจากขา TXD เป็นสัญญาณความต่างศักย์ (CANH, CANL) จุดเด่นคือมีฟังก์ชัน TXD Dominant Time-out ที่จะตัดการส่งข้อมูลทันทีหากขา TXD ค้าง เพื่อป้องกันไม่ให้โหนดนี้ทำระบบบัสล่มทั้งเครือข่าย

2. ภาครับสัญญาณ (Receiver): ทำหน้าที่อ่านค่าความต่างศักย์จากสายบัส ผ่านวงจรกรอง
3. สัญญาณรบกวน (Noise Filter) และแปลงกลับเป็นสัญญาณลอจิกส่งออกไปที่ขา RXD
4. วงจรควบคุม (Control Logic): รับคำสั่งจากขา S (Select) เพื่อสลับโหมดการทำงานระหว่าง High-Speed Mode (ทำงานปกติ) และ Silent Mode (ปิดภาคส่ง เพื่อฟังข้อมูลอย่างเดียว)
5. วงจรป้องกัน (Protection Circuits): มีระบบป้องกันสาย CAN ลัดวงจร (Short-Circuit Protection) และระบบตัดการทำงานอัตโนมัติเมื่อชิปมีอุณหภูมิสูงเกินไป (Thermal Shutdown)

2.6 SN65HVD230DR (3.3V CAN Transceiver)

2.6.1 SN65HVD230DR

SN65HVD230DR เป็นไอซีรับส่งสัญญาณ CAN Transceiver ซึ่งถูกออกแบบมาเพื่อใช้งานกับไมโคร คอนโทรลเลอร์ที่ใช้ระดับแรงดันไฟฟ้า 3.3V โดยเฉพาะ (เช่น ESP32, STM32 หรือ ARM Cortex รุ่นต่างๆ) ไอซีตัวนี้ทำหน้าที่เป็นตัวกลาง (Physical Layer) ระหว่างตัวควบคุมแคน (CAN Controller) และสายสัญญาณ CAN Bus ตามมาตรฐานอุตสาหกรรม ISO 11898-2 หน้าที่หลักคือการแปลงสัญญาณลอจิกดิจิทัล (TXD/RXD) จากไมโครคอนโทรลเลอร์ ให้กลายเป็นสัญญาณความต่างศักย์ (Differential Signal: CANH และ CANL) เพื่อส่งออกไปยังบัส และรับสัญญาณจากบัสแปลงกลับมาเป็นลอจิก 3.3V ให้ไมโครคอนโทรลเลอร์ประมวลผล โดยสามารถรองรับความเร็วในการสื่อสารได้สูงสุดถึง 1 Mbps

2.6.2 โครงสร้างภายในของ SN65HVD230DR

1. ภาควิส่งสัญญาณ (Transmitter/Driver): รับสัญญาณดิจิทัล 3.3V จากขา D มาแปลงเป็นสัญญาณความต่างศักย์ (CANH, CANL) เพื่อส่งลงบัส
2. ภาควิรับสัญญาณ (Receiver): อ่านค่าความต่างศักย์จากสายบัสผ่านวงจรเปรียบเทียบ (Comparator) ที่มีระบบป้องกันสัญญาณรบกวน (Hysteresis) และแปลงกลับเป็นลอจิก 3.3V ส่งออกที่ขา R
3. วงจรควบคุมโหมด (Mode Control Logic): ส่วนสำคัญที่เชื่อมต่อกับขา Rs ทำหน้าที่ปรับเปลี่ยนโหมดการทำงาน ได้แก่ การจำกัดความชันของสัญญาณ (Slope Control) เพื่อลดคลื่นแม่เหล็กไฟฟ้า (EMI) หรือสั่งปิดภาคส่งเพื่อเข้าสู่โหมดสแตนด์บาย (Standby Mode)

4. วงจรป้องกัน (Protection Circuits): มีระบบป้องกันสายบัลลัดวงจร (Short-Circuit Protection) ทนแรงดันกระชาก (Transient Protection) และระบบตัดการทำงานเมื่อความร้อนสูงเกิน (Thermal Shutdown)

5. วงจรสร้างแรงดันอ้างอิง (V_{ref}): จ่ายแรงดันไฟฟ้าอ้างอิงขนาด $V_{CC}/2$ ออกที่ขา V_{ref} เพื่อใช้ในการรักษาความเสถียรของระดับแรงดันบนสายเครือข่าย

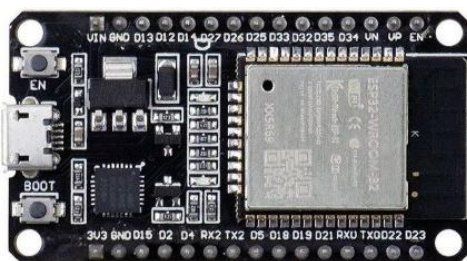


รูปที่ 11 SN65HVD230DR

2.7 ESP32 (ESP32 Development Board)

2.7.1 ESP32 (ESP32 Development Board)

เป็นบอร์ดที่ใช้ชิปประมวลผลตระกูล ESP32 ซึ่งพัฒนาโดยบริษัท Espressif Systems จุดเด่นของไมโครคอนโทรลเลอร์รุ่นนี้คือ เป็นชิปสถาปัตยกรรมแบบ 32 บิต Dual-Core (Xtensa 32-bit LX6) ที่มีความเร็วสัญญาณนาฬิกา (Clock Speed) สูงถึง 240 MHz ทำให้มีประสิทธิภาพในการประมวลผลข้อมูลที่ซับซ้อนได้อย่างรวดเร็ว นอกจากนี้ ESP32 ยังได้รับการออกแบบมาเพื่อรองรับระบบอินเทอร์เน็ตสรรพสิ่ง (IoT) โดยมีการติดตั้งวงจรสื่อสารไร้สายแบบครบวงจรไว้ในตัวชิป ทั้งเครือข่าย Wi-Fi (802.11 b/g/n) และ Bluetooth (รองรับทั้ง Classic Bluetooth และ Bluetooth Low Energy: BLE)



รูปที่ 12 ESP32 (ESP32 Development Board)

2.7.2 โครงสร้างและส่วนประกอบ

เนื่องจากตัวชิป ESP32 นิยมนำมาประกอบลงบนบอร์ดพัฒนา (Dev Module) เพื่ออำนวยความสะดวกในการทดลองและต่อวงจร โดยมีส่วนประกอบที่สำคัญบนบอร์ดดังนี้:

1. โมดูลประมวลผลหลัก (ESP-WROOM-32): เป็นโมดูลที่ครอบคลุมตัวชิป ESP32, หน่วยความจำแฟลช (Flash Memory), และสายอากาศ (PCB Antenna)
2. วงจรแปลงแรงดันไฟฟ้า (Voltage Regulator): ทำหน้าที่แปลงแรงดันไฟฟ้าจาก 5V (จากพอร์ต USB หรือแหล่งจ่ายไฟภายนอก) ให้เป็นแรงดัน 3.3V ซึ่งเป็นระดับแรงดันลอจิก (Logic Level) มาตรฐานที่ชิป ESP32 ใช้ในการทำงาน
3. ชิปแปลงสัญญาณ USB-to-Serial: (เช่น CP2102 หรือ CH340) ทำหน้าที่เชื่อมต่อการสื่อสารระหว่างไมโครคอนโทรลเลอร์กับคอมพิวเตอร์ เพื่อใช้นำเข้าโปรแกรมและดูผลการทำงานผ่าน Serial Monitor
4. ปุ่มควบคุม (Buttons): ประกอบด้วยปุ่ม EN (Enable/Reset) สำหรับเริ่มต้นการทำงานของบอร์ดใหม่ และปุ่ม BOOT สำหรับบังคับเข้าสู่โหมดนำเข้าโปรแกรม

2.7.3 ชุดควบคุมการสื่อสาร TWAI (Two-Wire Automotive Interface)

คุณสมบัติที่สำคัญมากของ ESP32 สำหรับโครงงานนี้คือ การมีชุดควบคุมโปรโตคอลการสื่อสารแบบแคน (CAN Controller) ติดตั้งมาให้ภายในตัวชิป โดยทางบริษัท Espressif Systems ใช้ชื่อเรียกว่า TWAI (Two-Wire Automotive Interface) เพื่อหลีกเลี่ยงปัญหาด้านลิขสิทธิ์เครื่องหมายการค้า แต่มีโครงสร้างและการทำงานตรงตามมาตรฐาน ISO 11898-1 (CAN Specification 2.0)

2.8 Raspberry Pi 4 Model B

2.8.1 Raspberry Pi 4 Model B

เป็นบอร์ดคอมพิวเตอร์ขนาดเล็ก (Single Board Computer: SBC) ที่พัฒนาโดยมูลนิธิ Raspberry Pi (Raspberry Pi Foundation) ซึ่งถูกออกแบบมาให้ทำงานได้เสมือนคอมพิวเตอร์เดสก์ท็อปขนาดย่อม โดยมีระบบปฏิบัติการเป็นของตัวเอง (เช่น Raspberry Pi OS ซึ่งมีพื้นฐานมาจาก Linux) การประมวลผลคือชิป Broadcom BCM2711 ซึ่งเป็นชิปแบบ System-on-Chip (SoC) สถาปัตยกรรม ARM Cortex-A72 แบบ 4 แกนประมวลผล (Quad-core) 64-bit ความเร็ว 1.5 GHz พร้อมกับหน่วยความจำหลัก (RAM) แบบ LPDDR4 ที่สามารถเลือกขนาดได้ (เช่น 2GB, 4GB หรือ 8GB) ทำให้มีความสามารถในการประมวลผลข้อมูลปริมาณมาก การจัดการฐานข้อมูล และการทำงานแบบหลายโปรแกรมพร้อมกัน (Multitasking) ได้อย่างมีประสิทธิภาพ



รูปที่ 13 Raspberry Pi 4 Model B

2.8.2 พอร์ตอินพุต/เอาต์พุตเนกประสงค์ (GPIO) และการเชื่อมต่อระบบแคน (CAN Bus Integration)

Raspberry Pi นิยมนำมาใช้ในงานโครงงานทางวิศวกรรมคือพอร์ต GPIO (General Purpose Input/Output) แบบ 40 ขา ซึ่งรองรับการเชื่อมต่อกับวงจรอิเล็กทรอนิกส์และโปรโตคอลการสื่อสารระดับฮาร์ดแวร์ เช่น UART, I2C และ SPI การประยุกต์ใช้ในระบบเครือข่าย CAN Bus: เนื่องจากไมโครโพรเซสเซอร์ของ Raspberry Pi ไม่มี ชุดควบคุม CAN (CAN Controller) ติดตั้งมาให้ในตัวเหมือนอย่างไมโครคอนโทรลเลอร์ ESP32 การจะให้ Raspberry Pi สื่อสารกับเครือข่าย CAN Bus ได้นั้น จึงจำเป็นต้องอาศัยอุปกรณ์ภายนอก ในทางปฏิบัติ การต่อวงจรจะใช้โปรโตคอล SPI ของ Raspberry Pi เชื่อมต่อเข้ากับไอซีควบคุมเครือข่ายแคน MCP2515 จากนั้นจึงส่งผ่านสัญญาณไปยังไอซีรับส่งสัญญาณแคน (CAN Transceiver) เช่น TJA1050 หรือ SN65HVD230 เพื่อเชื่อมต่อเข้ากับเครือข่าย CAN Bus ต่อไป โครงสร้างนี้ทำให้ Raspberry Pi สามารถทำหน้าที่เป็นตัวประมวลผลกลาง หรือเกตเวย์ (Gateway) ในการรวบรวมข้อมูลจากเซนเซอร์แต่ละโหนด



Function	Physical Pins				Function
	BCM	pin#	pin#	BCM	
3.3 Volts		1	2		5 Volts
GPIO/SODAI (I2C)	2	3	4		5 Volts
GPIO/SCL1 (I2C)	3	5	6		GND
GPIO/GCLK	4	7	8	14	TX UART/GPIO
GND		9	10	15	RX UART/GPIO
GPIO	17	11	12	18	GPIO
GPIO	27	13	14		GND
GPIO	22	15	16	23	GPIO
3.3 Volts		17	18	24	GPIO
MOSI (SPI)	10	19	20		GND
MISO (SPI)	9	21	22	25	GPIO
SCLK (SPI)	11	23	24	8	CEO_N (SPI)
GND		25	26	7	CE1_N (SPI)
RESERVED		27	28		RESERVED
GPIO	5	29	30		GND
GPIO	6	31	32	12	GPIO
GPIO	13	33	34		GND
GPIO	19	35	36	16	GPIO
GPIO	26	37	38	20	GPIO
GND		39	40	21	GPIO

รูปที่ 14 GPIO ของ Raspberry Pi 4 Model B

2.9 TFT LCD with ILI9341

2.9.1 TFT LCD

เป็นอุปกรณ์แสดงผลแบบสี (Color Display) ที่ได้รับความนิยมในงานระบบสมองกลฝังตัว หน้าจอมีความละเอียดระดับ QVGA คือ 320x240 พิกเซล และสามารถแสดงผลสีได้สูงสุด 65,536 สี (16-bit Color) ทำให้สามารถแสดงผลกราฟิก ข้อความ ไอคอน หรือรูปภาพได้อย่างคมชัด

2.9.2 ILI9341

ทำหน้าที่เป็นตัวกลางรับคำสั่งและข้อมูลภาพจากไมโครคอนโทรลเลอร์ (เช่น Arduino, ESP32 หรือ Raspberry Pi) ไปแปลงเป็นสัญญาณไฟฟ้าเพื่อควบคุมการเปล่งแสงของแต่ละพิกเซลบนหน้าจอ ไอซี ILI9341 มีหน่วยความจำแสดงผล (Video RAM หรือ GRAM) ในตัวขนาด 172,800 ไบต์ (รองรับภาพขนาด 240x320 จุด ที่ความละเอียด 18-bit) และจุดเด่นที่สำคัญคือรองรับการสื่อสารข้อมูลหลายรูปแบบ โดยในโมดูลส่วนใหญ่มักจะออกแบบให้สื่อสารผ่านโปรโตคอล SPI (Serial Peripheral Interface) ซึ่งช่วยลดจำนวนขาสัญญาณที่ต้องเชื่อมต่อกับไมโครคอนโทรลเลอร์



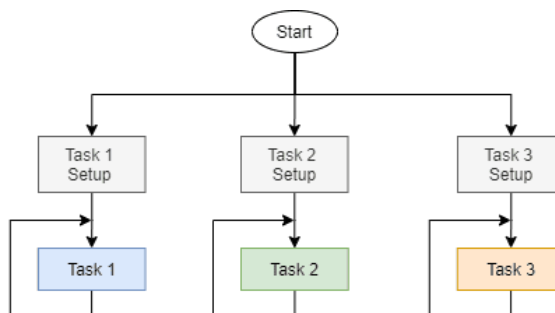
รูปที่ 15 TFT LCD

2.10 FreeRTOS (Free Real-Time Operating System)

2.10.1 FreeRTOS (Free Real-Time Operating System)

FreeRTOS เป็นระบบปฏิบัติการแบบเวลาจริง (Real-Time Operating System: RTOS) แบบโอเพนซอร์สที่ถูกออกแบบมาสำหรับไมโครคอนโทรลเลอร์ขนาดเล็กโดยเฉพาะ หน้าที่หลักของระบบปฏิบัติการประเภทนี้คือ การบริหารจัดการทรัพยากรของซีพียูและเวลาในการประมวลผล เพื่อให้ระบบสามารถตอบสนองต่อเหตุการณ์ต่างๆ ได้อย่างรวดเร็วและสามารถคาดเดา

เวลาทำงานได้ (Deterministic) ในการเขียนโปรแกรมแบบดั้งเดิม (Bare-metal หรือ Super Loop) คำสั่งจะถูกประมวลผลเรียงตามลำดับในลูปหลักเพียงลูปเดียว หากมีฟังก์ชันใดทำงานล่าช้า จะส่งผลกระทบต่อการทำงานของทั้งระบบ แต่เมื่อใช้ FreeRTOS โปรแกรมจะถูกแบ่งออกเป็นส่วนย่อย ๆ ที่ทำงานเป็นอิสระต่อกัน เรียกว่า Task โดยมีตัวจัดลำดับงาน (Scheduler) ทำหน้าที่สลับการทำงานของแต่ละ Task ตามลำดับความสำคัญ (Priority) ทำให้ไมโครคอนโทรลเลอร์สามารถทำงานหลายอย่างเสมือนทำงานพร้อมกันได้ (Multitasking)



รูปที่ 16 การทำงานของ RTOS (Real-Time Operating System)

2.10.2 FreeRtos on ESP32

ไมโครคอนโทรลเลอร์ทั่วไปมักจะมีแกนประมวลผลเพียงแกนเดียว (Single-Core) แต่ชิปตระกูล ESP32 มีสถาปัตยกรรมแบบ 2 แกนประมวลผล (Dual-Core) ได้แก่:

1. Core 0: มักถูกเรียกว่า Protocol CPU (PRO_CPU) ทำหน้าที่จัดการงานที่เกี่ยวข้องกับโครงข่ายและการสื่อสารระดับฮาร์ดแวร์
2. Core 1: มักถูกเรียกว่า Application CPU (APP_CPU) ทำหน้าที่ประมวลผลโค้ดโปรแกรมทั่วไปของผู้ใช้งาน

2.10.3 องค์ประกอบหลักในการจัดการทรัพยากร (Key Core Components)

เพื่อให้ Task ทำงานแยกอิสระสามารถแลกเปลี่ยนข้อมูลและใช้งานทรัพยากรฮาร์ดแวร์ร่วมกันได้อย่างปลอดภัย (Thread-safe) FreeRTOS มีเครื่องมือทางทฤษฎีที่สำคัญ ดังนี้

1. คิว (Queues): เป็นโครงสร้างข้อมูลพื้นฐานที่ใช้ในการส่งผ่านข้อมูลระหว่าง Task หรือระหว่าง Interrupt กับ Task โดยทำงานในรูปแบบเข้าก่อนออกก่อน (FIFO - First In First Out) ช่วยป้องกันปัญหาการอ่าน/เขียนตัวแปรพร้อมกันจนข้อมูลเสียหาย
2. เซมาฟอว์ (Semaphores): เป็นกลไกที่ใช้สำหรับซิงโครไนซ์ (Synchronize) การทำงานระหว่าง Task เช่น การแจ้งเตือน Task หนึ่งให้ตื่นขึ้นมาทำงานเมื่อมี

เหตุการณ์บางอย่างเกิดขึ้น (Binary Semaphore) หรือใช้จำกัดจำนวนการเข้าถึงทรัพยากร (Counting Semaphore)

3. มิวเทกซ์ (Mutexes): ย่อมาจาก Mutual Exclusion มีลักษณะคล้าย Semaphore แต่ถูกออกแบบมาเพื่อ "ล็อก" ทรัพยากรที่ต้องใช้งานร่วมกัน (Shared Resource) โดยเฉพาะ เมื่อ Task ใดครอบครอง Mutex อยู่ Task อื่นจะไม่สามารถเข้าใช้งานทรัพยากรนั้นได้

บทที่ 3

หลักการทำงานและการออกแบบ

3.1 System Architecture

ระบบ Voice-Controlled Automotive System ถูกออกแบบเพื่อให้ผู้ใช้สามารถควบคุมฟังก์ชันต่าง ๆ ภายในรถยนต์ผ่านคำสั่งเสียง โดยระบบจะรับสัญญาณเสียงจากไมโครโฟน จากนั้นทำการวิเคราะห์คำสั่งเสียงและแปลงคำสั่งดังกล่าวให้อยู่ในรูปแบบ CAN message เพื่อส่งไปควบคุมอุปกรณ์ภายในรถยนต์ผ่านเครือข่าย Controller Area Network (CAN Bus) การทำงานของระบบเริ่มจากการตรวจจับคำปลุก (Wake Word Detection) เพื่อเปิดการทำงานของระบบ เมื่อระบบตรวจพบคำปลุกแล้วจะเริ่มรับคำสั่งเสียงของผู้ใช้และแปลงสัญญาณเสียงเป็นข้อความด้วยเทคนิค Speech-to-Text (STT) จากนั้นข้อความที่ได้จะถูกนำไปวิเคราะห์ความหมายของคำสั่งในขั้นตอน Intent Parsing หลังจากระบบสามารถระบุประเภทของคำสั่งได้แล้ว จะทำการแปลงคำสั่งดังกล่าวให้เป็น CAN frame ที่เหมาะสมกับอุปกรณ์ที่ต้องการควบคุม เช่น ระบบปรับอากาศ กระจกไฟฟ้า หรือไฟภายในรถยนต์ และส่งข้อมูลผ่าน CAN Bus ไปยังอุปกรณ์ปลายทางเพื่อดำเนินการตามคำสั่ง

3.2 Wake Word Detection

Wake Word Detection เป็นขั้นตอนที่ใช้สำหรับตรวจจับคำปลุก (Wake Word) เพื่อเริ่มต้นการทำงานของระบบสั่งงานด้วยเสียง โดยระบบจะอยู่ในสถานะรอรับสัญญาณเสียงจากไมโครโฟนตลอดเวลา และจะเริ่มกระบวนการรับคำสั่งเสียงเมื่อพบคำปลุกที่กำหนดไว้ เช่น “Hello Car” ในระบบนี้ใช้โมเดล Vosk Speech Recognition สำหรับการตรวจจับคำปลุก โดยสัญญาณเสียงจากไมโครโฟนจะถูกอ่านด้วยอัตรา Sampling ที่ 48 kHz และทำการแปลงเป็น 16 kHz เพื่อให้เหมาะสมกับการประมวลผลของโมเดล ก่อนนำไปวิเคราะห์ข้อความที่ได้จากการรู้จำเสียง นอกจากนี้ระบบยังใช้เทคนิค Voice Activity Detection (VAD) เพื่อช่วยตรวจสอบว่ามีสัญญาณเสียงพูดโดยจะพิจารณาจากระดับพลังงานของสัญญาณเสียง หากพบว่ามีความถี่เสียงพูด ระบบจะทำการประมวลผลข้อมูลเสียงและตรวจสอบความใกล้เคียงกับคำปลุกที่กำหนดไว้

3.3 Speech Recognition

Speech Recognition เป็นขั้นตอนที่ใช้ในการแปลงสัญญาณเสียงของผู้ใช้ให้เป็นข้อความ (Speech-to-Text) เพื่อให้ระบบสามารถนำข้อมูลไปวิเคราะห์คำสั่งได้ ในระบบนี้ เมื่อ Wake Word ถูกตรวจพบแล้ว ระบบจะเริ่มบันทึกเสียงคำสั่งของผู้ใช้ผ่านไมโครโฟน สัญญาณเสียงที่ได้รับจะถูกประมวลผลโดยใช้ไลบรารี SpeechRecognition ซึ่งทำหน้าที่แปลงเสียงพูดเป็นข้อความโดยใช้บริการรู้จำเสียงพูดจากภายนอก โดยระบบจะทำการวิเคราะห์สัญญาณเสียงและสร้างผลลัพธ์เป็นข้อความที่ตรงกับคำพูดของผู้ใช้มากที่สุด หลังจากกระบวนการ Speech-to-Text เสร็จสิ้น ระบบจะ

ได้ข้อความของคำสั่งเสียง เช่น การเปิดหรือปิดเครื่องปรับอากาศ การปรับอุณหภูมิ หรือการควบคุมกระจกหน้าต่าง ข้อความดังกล่าวจะถูกส่งต่อไปยังขั้นตอน Intent Parsing เพื่อวิเคราะห์ความหมายของคำสั่งและกำหนดการทำงานของระบบในขั้นตอนถัดไป

3.4 Intent Parsing

Intent Parsing เป็นขั้นตอนที่ใช้ในการวิเคราะห์ความหมายของข้อความที่ได้จากกระบวนการ Speech Recognition เพื่อระบุว่าผู้ใช้ต้องการให้ระบบดำเนินการใด โดยในขั้นตอนนี้ระบบจะทำการแปลงข้อความคำสั่งเสียงให้อยู่ในรูปแบบคำสั่งเชิงโครงสร้าง (Structured Command) ซึ่งประกอบด้วย Intent และ Parameter ที่เกี่ยวข้อง Intent หมายถึงประเภทของคำสั่งที่ผู้ใช้ต้องการ เช่น การเปิดเครื่องปรับอากาศ การปรับอุณหภูมิ การเปิดหรือปิดกระจกหน้าต่าง หรือการควบคุมไฟภายในรถ ส่วน Parameter คือค่าที่ใช้กำหนดรายละเอียดเพิ่มเติมของคำสั่ง เช่น อุณหภูมิที่ต้องการ ระดับความแรงของพัดลม หรือเปอร์เซ็นต์การเปิดกระจก ในระบบนี้มีการออกแบบ Intent Parsing ให้รองรับคำสั่งหลายประเภท เช่น

1. การควบคุมระบบปรับอากาศ (HVAC Control)
2. การควบคุมกระจกหน้าต่าง (Window Control)
3. การควบคุมไฟภายในรถ (Cabin Light Control)

ตารางที่ 2 Intent ที่รองรับในระบบ

Intent	หน้าที่ของระบบ	ตัวอย่างคำสั่งเสียง
AC_ON	เปิดระบบเครื่องปรับอากาศของรถ	"เปิดแอร์", "Turn on AC"
AC_OFF	ปิดระบบเครื่องปรับอากาศ	"ปิดแอร์", "Turn off AC"
SET_TEMP	ตั้งค่าอุณหภูมิแอร์เป็นค่าที่กำหนด	"ตั้งแอร์ 24 องศา", "Set temperature to 24"
HVAC_TEMP_UP	เพิ่มอุณหภูมิแอร์ขึ้นที่ระดับ	"เพิ่มอุณหภูมิ", "Temperature up"
HVAC_TEMP_DOWN	ลดอุณหภูมิแอร์ลงที่ระดับ	"ลดอุณหภูมิ", "Temperature down"
HVAC_FAN_UP	เพิ่มระดับความแรงพัดลม	"เพิ่มพัดลม", "Fan speed up"
HVAC_FAN_DOWN	ลดระดับความแรงพัดลม	"ลดพัดลม", "Fan speed down"
HVAC_FAN_SET	ตั้งระดับพัดลมตามค่าที่กำหนด	"ตั้งพัดลมระดับ 3", "Set fan speed to 3"

Intent	หน้าที่ของระบบ	ตัวอย่างคำสั่งเสียง
WINDOW_OPEN	เปิดกระจกรถ	"เปิดกระจก", "Open window"
WINDOW_CLOSE	ปิดกระจกรถ	"ปิดกระจก", "Close window"
WINDOW_SET_POSITION	ตั้งตำแหน่งกระจก (เช่น 50%)	"เปิดกระจกครึ่งหนึ่ง", "Set window to 50 percent"
CABIN_LIGHT_ON	เปิดไฟในห้องโดยสาร	"เปิดไฟในรถ", "Turn on cabin light"
CABIN_LIGHT_OFF	ปิดไฟในห้องโดยสาร	"ปิดไฟในรถ", "Turn off cabin light"

3.4.1 Hybrid Intent Parsing Architecture

ระบบนี้ใช้โครงสร้าง Hybrid Intent Parsing ซึ่งประกอบด้วยสองวิธี ได้แก่

1. Local Rule-Based Parser
2. AI-based Parser (GPT)

โดยระบบจะใช้ Local Parser เป็นขั้นตอนแรก หากไม่สามารถตีความคำสั่งได้ ระบบจะส่งข้อความไปวิเคราะห์ด้วยโมเดล AI เพื่อเพิ่มความยืดหยุ่นในการรองรับคำสั่งที่หลากหลาย แนวทางนี้ช่วยลดเวลาในการประมวลผลและลดการใช้ทรัพยากรของโมเดล AI ในกรณีที่คำสั่งสามารถตีความได้ด้วยกฎที่กำหนดไว้

3.4.2 Local Intent Parser

Local Intent Parser เป็นโมดูลที่ใช้กฎ (Rule-based Parsing) ในการตรวจสอบข้อความคำสั่ง โดยใช้เทคนิคการจับคำสำคัญ (Keyword Matching) และการวิเคราะห์ค่าตัวเลขในข้อความ เพื่อระบุประเภทของคำสั่งและค่าพารามิเตอร์ที่เกี่ยวข้อง

ตัวอย่าง เช่น

1. คำว่า “เปิดแอร์” หรือ “turn on AC” จะถูกตีความเป็น Intent AC_ON
2. คำว่า “ตั้งแอร์ 25 องศา” จะถูกตีความเป็น Intent SET_TEMP พร้อมค่า

อุณหภูมิ 25

3. คำว่า “เปิดกระจกหน้า 50 เปอร์เซ็นต์” จะถูกตีความเป็น

WINDOW_SET_POSITION

4. ระบบยังมีการตรวจสอบและจำกัดค่าพารามิเตอร์ให้อยู่ในช่วงที่เหมาะสม เช่น
5. อุณหภูมิแอร์: 16 – 30 °C

6. ระดับพัลล: 0 – 7
7. ระดับการเปิดกระจก: 0 – 100 %
8. การตรวจสอบช่วงค่าดังกล่าวช่วยป้องกันการส่งคำสั่งที่ไม่ถูกต้องไปยังระบบ

ควบคุมรถยนต์

3.4.3 AI-based Intent Parser

ในกรณีที่ Local Parser ไม่สามารถตีความคำสั่งได้ ระบบจะใช้ AI-based Intent Parser ซึ่งใช้โมเดลภาษา (Large Language Model) เพื่อวิเคราะห์ความหมายของข้อความคำสั่ง โมเดลจะรับข้อความคำสั่งจากผู้ใช้และทำการแปลงให้อยู่ในรูปแบบ JSON structure ที่กำหนดไว้ โดยจะมีการกำหนดรายการ Intent ที่อนุญาตและกฎในการตีความคำสั่ง เพื่อให้ผลลัพธ์มีรูปแบบที่สอดคล้องกับระบบควบคุม

3.4.4 Intent Output Structure

หลังจากขั้นตอน Intent Parsing ระบบจะจัดรูปแบบผลลัพธ์ของคำสั่งให้อยู่ในรูปแบบข้อมูลเชิงโครงสร้าง (Structured Data) เพื่อให้สามารถนำไปใช้ในขั้นตอนถัดไปของระบบได้อย่างเป็นระบบ โดยโครงสร้างข้อมูลนี้เรียกว่า Intent Output Structure ซึ่งทำหน้าที่เป็นตัวกลางระหว่างขั้นตอนการตีความคำสั่งเสียงและขั้นตอนการสร้าง CAN message

Intent Output Structure ถูกออกแบบให้อยู่ในรูปแบบ dictionary หรือ JSON structure ซึ่งช่วยให้ระบบสามารถจัดเก็บข้อมูลของคำสั่งและพารามิเตอร์ที่เกี่ยวข้องได้อย่างชัดเจน โครงสร้างข้อมูลหลักประกอบด้วยฟิลด์สำคัญ ได้แก่ intent, value, และ window ตัวอย่างเช่น หากผู้ใช้พูดว่า “ตั้งแอร์ 25 องศา” ระบบจะสร้างโครงสร้างข้อมูล ดังนี้

```
{
  "intent": "SET_TEMP",
  "value": 25
}
```

3.5 Intent-to-CAN Mapping

Intent-to-CAN Mapping เป็นขั้นตอนที่ทำหน้าที่แปลงผลลัพธ์จาก Intent Parsing ให้เป็นคำสั่งที่สามารถส่งผ่านเครือข่าย Controller Area Network (CAN Bus) ได้ โดยระบบจะนำข้อมูลจาก Intent Output Structure เช่น intent, value และ window มาแปลงเป็น CAN message ซึ่งประกอบด้วย CAN Identifier (CAN ID) และ CAN Data Field ระบบนี้ใช้ Standard CAN Frame แบบ 11-bit identifier และข้อมูล 8 byte ต่อเฟรม ซึ่งเป็นรูปแบบที่นิยมใช้ในระบบอิเล็กทรอนิกส์ของยานยนต์ โดยแต่ละอุปกรณ์ภายในรถจะถูกกำหนด CAN ID เฉพาะเพื่อใช้ระบุ

ประเภทของข้อความ จากการออกแบบระบบ CAN ของโปรเจกต์นี้ มีการกำหนด CAN ID สำหรับอุปกรณ์หลักดังตารางที่ 3

ตารางที่ 3 CAN ID สำหรับอุปกรณ์หลัก

System	CAN ID	Description
HVAC Command	0x245	คำสั่งควบคุมระบบปรับอากาศ
HVAC Status	0x246	สถานะของระบบปรับอากาศ
Window Command	0x325	คำสั่งควบคุมกระจกหน้าต่าง
Window Status	0x326	สถานะของกระจก
Cabin Light Command	0x420	คำสั่งควบคุมไฟภายในรถ
Cabin Light Status	0x421	สถานะไฟภายในรถ

3.5.1 CAN Data Field Structure

CAN Data Field มีขนาด 8 Byte โดยแต่ละ byte ถูกออกแบบให้มีหน้าที่เฉพาะ เช่น การระบุประเภทคำสั่ง ค่าพารามิเตอร์ และข้อมูลตรวจสอบความถูกต้องของเฟรม

ตารางที่ 4 HVAC Command Frame

Byte	Field	Description
Byte0	Command Type	ประเภทคำสั่ง HVAC
Byte1	Temperature	ค่าอุณหภูมิ
Byte2	Fan Level	ระดับพัดลม
Byte3	Mode Flags	โหมด AC
Byte4	Reserved	สำรอง
Byte5	Rolling Counter	ตัวนับเฟรม
Byte6	CRC8	ตรวจสอบข้อมูล
Byte7	Alive Pattern	0xAA

ตารางที่ 5 Window Command Frame

Byte	Field	Description
Byte0	Window Mask	ตำแหน่งกระจก
Byte1	Position	เปอร์เซ็นต์การเปิด
Byte2	Speed	ความเร็วการเคลื่อน
Byte3	Command	Window Move
Byte4	Reserved	สำรอง
Byte5	Rolling Counter	ตัวนับเฟรม
Byte6	CRC8	ตรวจสอบข้อมูล
Byte7	Alive Pattern	0xAA

ตารางที่ 6 Light Command Frame

Byte	Field	Description
Byte0	Mode	โหมด
Byte1	Brightness	เปอร์เซ็นต์การเปิด
Byte2	Reserved	สำรอง
Byte3	Reserved	สำรอง
Byte4	Reserved	สำรอง
Byte5	Rolling Counter	ตัวนับเฟรม
Byte6	CRC8	ตรวจสอบข้อมูล
Byte7	Alive Pattern	0xAA

3.5.2 Rolling Counter

เพื่อเพิ่มความน่าเชื่อถือของการสื่อสาร CAN ระบบใช้ Rolling Counter ซึ่งเป็นตัวนับค่าที่เพิ่มขึ้นทุกครั้งที่มีการส่ง CAN message โดยค่าจะอยู่ในช่วง 0-15 และจะวนกลับไปเริ่มใหม่เมื่อถึงค่าสูงสุด Rolling Counter ถูกเก็บไว้ใน Byte5 ของ CAN data

3.5.3 CRC8 Error Detection

ระบบยังใช้ CRC8 checksum เพื่อตรวจสอบความถูกต้องของข้อมูลก่อนส่ง โดยใช้ polynomial ของมาตรฐาน CRC8 SAE J180 (0x1D) ค่า CRC จะถูกคำนวณจาก Byte0 – Byte5 ของ CAN frame และเก็บไว้ใน Byte6

ตัวอย่าง Intent to CAN Mapping

ตัวอย่างที่ 1 คำสั่งเสียง: “เปิดแอร์”

Intent Output:

```
{
    "intent": "AC_ON",
}
```

CAN Frame

Field	Value
CAN ID	0x245
DATA	[0x01, 0x41, 0x03, 0x01, 0x00, 0x02, CRC, 0xAA]

ตารางที่ 7 คำอธิบาย

Byte	Value	Description
Byte0	0x01	Command: AC ON
Byte1	0x41	Temperature (25°C + 40 offset)
Byte2	0x03	Fan Level
Byte3	0x01	Mode
Byte4	0x00	Reserved
Byte5	0x02	Rolling Counter
Byte6	CRC	CRC8 Checksum
Byte7	0xAA	Alive Pattern

ตัวอย่างที่ 2 คำสั่งเสียง : “เปิดกระจกหน้าซ้าย”

Intent Output:

```
{
    "intent": "WINDOW_OPEN",
    "window": "WINDOW_FRONT_LEFT"
}
```

CAN Frame

Field	Value
CAN ID	0x325
DATA	[0x01, 0x64, 0x05, 0x01, 0x00, 0x03, CRC, 0xAA]

ตารางที่ 8 คำอธิบาย

Byte	Value	Description
Byte0	0x01	Window Mask (Front Left)
Byte1	0x64	Position 100%
Byte2	0x05	Speed
Byte3	0x01	Command: Open
Byte4	0x00	Reserved
Byte5	0x03	Rolling Counter
Byte6	CRC	CRC8 Checksum
Byte7	0xAA	Alive Pattern

ตัวอย่างที่ 3 คำสั่งเสียง : “เปิดไฟในรถ”

Intent Output:

```
{
    "intent": "CABIN_LIGHT_ON"
}
```

CAN Frame

Field	Value
CAN ID	0x420
DATA	[0x01, 0x64, 0x00, 0x00, 0x00, 0x04, CRC, 0xAA]

ตารางที่ 9 คำอธิบาย

Byte	Value	Description
Byte0	0x01	Light ON
Byte1	0x64	Brightness 100%
Byte2	0x00	Reserved
Byte3	0x00	Reserved
Byte4	0x00	Reserved
Byte5	0x04	Rolling Counter
Byte6	CRC	CRC8 Checksum
Byte7	0xAA	Alive Pattern

ตัวอย่างที่ 4 คำสั่งเสียง : “ตั้งอุณหภูมิแอร์ 24 องศา”

Intent Output:

```
{
    "intent": "SET_TEMP",
    "value": 24
}
```

CAN Frame

Field	Value
CAN ID	0x245
DATA	[0x01, 0x40, 0x03, 0x01, 0x00, 0x05, CRC, 0xAA]

ตารางที่ 10 คำอธิบาย

Byte	Value	Description
Byte0	0x01	Command: SET TEMP
Byte1	0x40	Temperature = 24°C
Byte2	0x03	Fan Level
Byte3	0x01	HVAC Mode
Byte4	0x00	Reserved
Byte5	0x05	Rolling Counter
Byte6	CRC	CRC8 Checksum
Byte7	0xAA	Alive Pattern

3.6 CAN Message Transmission

3.6.1 CAN Message Transmission

เป็นขั้นตอนที่ใช้ในการส่งคำสั่งควบคุมจากระบบประมวลผลเสียงไปยังอุปกรณ์ภายในรถยนต์ผ่าน เครือข่าย Controller Area Network (CAN Bus) หลังจากขั้นตอน Intent-to-CAN Mapping สร้างข้อมูลคำสั่งในรูปแบบ CAN frame แล้ว ระบบจะทำการส่งข้อมูลดังกล่าวผ่าน CAN interface เพื่อให้ ECU หรืออุปกรณ์ปลายทางดำเนินการตามคำสั่งที่ได้รับ ในระบบนี้ใช้ Raspberry Pi เป็นหน่วยประมวลผลหลัก และใช้ SocketCAN interface สำหรับการสื่อสารกับ CAN controller โดยการส่งข้อมูล CAN message ถูกดำเนินการผ่านไลบรารี python-can ก่อน `slink set can0 up type can bitrate 125000` คำสั่งนี้จะกำหนดให้ CAN interface can0 ทำงานที่ bitrate 125 kbps ซึ่งเป็นอัตราการส่งข้อมูลที่ใช้ในระบบของโปรเจกต์นี้

3.6.2 CAN Message Construction

การสร้าง CAN message จะกำหนดค่า CAN ID และ Data Field ก่อนส่งออกไปยัง CAN bus โดยใช้โครงสร้างข้อมูลของไลบรารี python-can

CAN message ประกอบด้วย

1. CAN Identifier (CAN ID) – ใช้ระบุประเภทของข้อความ
2. Data Length Code (DLC) – จำนวน byte ของข้อมูล
3. Data Field – ข้อมูลคำสั่งขนาด 8 byte

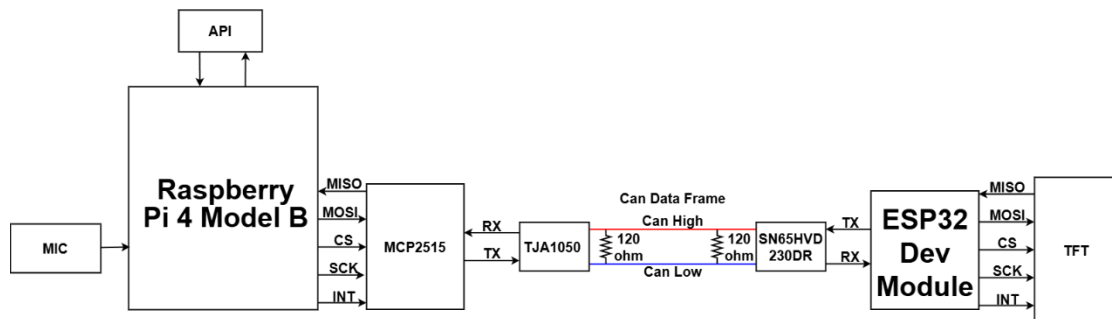
ตัวอย่างการสร้าง CAN message

```
msg = can.Message(arbitration_id = 0x245, data = [0x01, 0x40, 0x03, 0x01, 0x00, 0x05, 0x9C, 0xAA], is_extended_id = False)
```

ในตัวอย่างนี้

1. arbitration_id คือ CAN ID
2. data คือข้อมูล 8-byte ที่ประกอบด้วยคำสั่งควบคุม
3. is_extended_id=False หมายถึงใช้ Standard CAN frame (11-bit ID)

3.7 Block Diagram Hardware (Schematic)



รูปที่ 17 Block Diagram Hardware

3.7.1 โหนดประมวลผลกลาง (Master Node: Raspberry Pi 4 Model B)

โหนดนี้ทำหน้าที่เป็นศูนย์กลางในการรวบรวมและประมวลผลข้อมูลของระบบ เนื่องจาก Raspberry Pi ไม่มีชุดควบคุมแคน (CAN Controller) ในตัว จึงต้องอาศัยอุปกรณ์เชื่อมต่อภายนอก โดยมีรูปแบบการทำงานดังนี้

1. การเชื่อมต่อผ่าน SPI: Raspberry Pi จะสื่อสารกับไอซีควบคุมแคน MCP2515 ผ่านโปรโตคอล SPI โดยใช้สายสัญญาณหลัก 4 เส้น ได้แก่ MISO, MOSI, SCK และ CS นอกจากนี้ยังมีการต่อสาย INT (Interrupt) เพิ่มเติม เพื่อให้ MCP2515 สามารถส่งสัญญาณแจ้งเตือน

ไปยัง Raspberry Pi ได้ทันทีเมื่อมีข้อมูลใหม่เข้ามาในระบบ ช่วยลดภาระการประมวลผลที่ต้องคอยวนลูปตรวจสอบข้อมูล (Polling)

2. การแปลงสัญญาณสื่อบัส: MCP2515 จะจัดการโปรโตคอลแคนและส่งข้อมูลระดับลอจิก (TX/RX) ไปยังไอซีรับส่งสัญญาณแคนเบอร์ TJA1050 ซึ่งจะทำหน้าที่แปลงลอจิก 5V ให้เป็นสัญญาณความต่างศักย์เพื่อส่งออกไปยังสาย CAN High และ CAN Low

3.7.2 CAN Bus Physical Layer

สายสัญญาณส่วนกลางที่ใช้เชื่อมต่อระหว่างโหนด ทำงานตามมาตรฐาน ISO 11898-2 (High-Speed CAN) ประกอบด้วยสายสัญญาณ 2 เส้น คือ CAN High และ CAN Low Terminating Resistors: ในการออกแบบได้ทำการติดตั้งตัวต้านทานขนาด 120 โอห์ม คร่อมระหว่างสาย CAN High และ CAN Low ที่ปลายทั้งสองด้านของเครือข่าย (ฝั่ง TJA1050 และฝั่ง SN65HVD230DR) ซึ่งเป็นสิ่งจำเป็นอย่างยิ่งในการสื่อสารความเร็วสูง เพื่อป้องกันปัญหาการสะท้อนกลับของสัญญาณ (Signal Reflection) ที่ปลายสาย ซึ่งอาจทำให้ข้อมูลเกิดความผิดพลาดได้

3.7.3 Local Node: ESP32 Dev Module และหน้าจอ TFT

โหนดนี้ทำหน้าที่รับข้อมูลจากเครือข่ายมาแสดงผลให้ผู้ใช้งานทราบหรือส่งข้อมูลสถานะกลับไปยังศูนย์กลาง โดยใช้ไมโครคอนโทรลเลอร์ ESP32 ซึ่งมีหลักการทำงานดังนี้

1. การเชื่อมต่อกับเครือข่ายแคน: เนื่องจาก ESP32 มีชุดควบคุมแคน (TWAI Controller) จึงสามารถต่อขาสัญญาณ TX และ RX จากไมโครคอนโทรลเลอร์เข้ากับไอซีรับส่งสัญญาณแคนเบอร์ SN65HVD230DR ได้โดยตรง ซึ่งไอซีเบอร์นี้รองรับระดับแรงดัน 3.3V ทำให้เข้ากันได้กับลอจิกของ ESP32 อย่างสมบูรณ์โดยไม่ต้องใช้วงจรแปลงแรงดัน

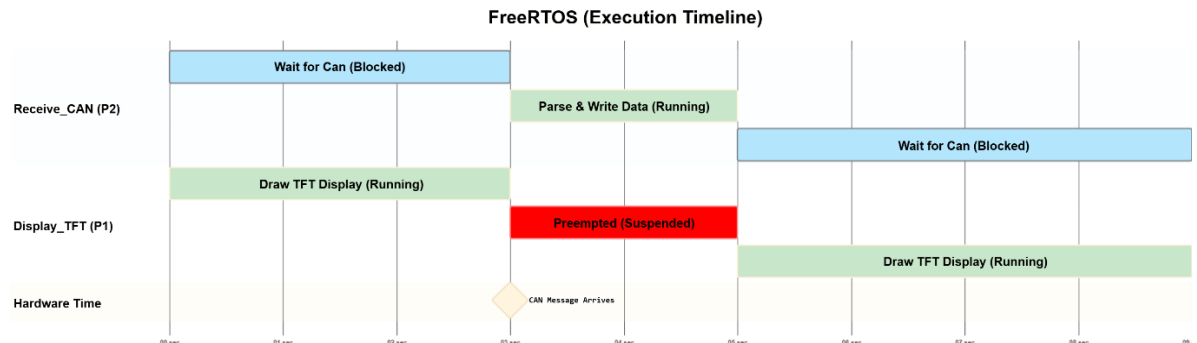
2. การแสดงผล TFT: เมื่อ ESP32 ได้รับข้อมูลจาก CAN Bus จะนำข้อมูลนั้นไปประมวลผลและส่งไปแสดงผลที่หน้าจอ TFT LCD โดยใช้การสื่อสารผ่านโปรโตคอล SPI (MISO, MOSI, CS, SCK) ซึ่งเป็นการใช้ขาสัญญาณชุดแยกต่างหากจากการสื่อสารของระบบ CAN ทำให้ไมโครคอนโทรลเลอร์สามารถจัดการงานทั้งสองส่วนได้อย่างมีประสิทธิภาพ

3.8 โครงสร้างสถาปัตยกรรมซอฟต์แวร์ (Software Architecture)

การพัฒนาซอฟต์แวร์สำหรับบอร์ด ESP32 ในโครงงานนี้ ได้ประยุกต์ใช้ระบบปฏิบัติการแบบเวลาจริง (FreeRTOS) เป็นแกนหลักในการขับเคลื่อนระบบ เพื่อให้การรับส่งข้อมูล CAN Bus และการอัปเดตหน้าจอ TFT สามารถทำงานขนานกันไปได้อย่างไร้รอยต่อ (Multitasking) แต่จะทำการสร้างงาน (Task) แยกกัน 2 ส่วนหลัก

1. Task "Recive_Can": กำหนดความสำคัญ (Priority) ไว้ที่ระดับ 2 (สูงกว่า) ทำหน้าที่ดักจับข้อมูลจากเครือข่ายตลอดเวลา

2. Task "Display_TFT": กำหนดความสำคัญไว้ที่ระดับ 1 ทำหน้าที่จัดการหน้าจอแสดงผลตามข้อมูลที่ได้รับมา



รูปที่ 18 การทำงานของ FreeRTOS บนบอร์ด ESP32 ในรูปแบบของ Execution Timeline

โดยทั้งสอง Task จะแลกเปลี่ยนข้อมูลกันผ่านโครงสร้างข้อมูล (Data Structure) ชื่อ currentCarState (ประเภท CarData) ซึ่งถูกประกาศเป็นตัวแปรแบบ volatile เพื่อป้องกันไม่ให้คอมไพเลอร์ปรับแต่งค่าการอ่านเขียนในหน่วยความจำ รับประกันว่าข้อมูลสถานะรถยนต์จะเป็นข้อมูลล่าสุดเสมอ

3.9 ระบบการรับข้อมูลจาก CAN Bus (TWAI Controller Integration)

ระบบได้ประยุกต์ใช้ชุดคำสั่งควบคุม TWAI (Two-Wire Automotive Interface) ซึ่งเป็นฮาร์ดแวร์ควบคุม CAN Bus ที่ฝังอยู่ภายในตัวไมโครคอนโทรลเลอร์ ESP32 โดยมีการกำหนดค่าและหลักการทำงานแบ่งออกเป็น 2 ส่วนหลัก

3.9.1 Hardware Configuration

ไอซีแปลงสัญญาณ (CAN Transceiver) ระบบถูกตั้งค่าให้ทำงานที่ความเร็วบอดเรต (Baud Rate) 125 kbps และ ตั้งค่าตัวกรองสัญญาณ ให้รับข้อความทุกรูปแบบที่อยู่บนเครือข่ายระบบดักจับข้อมูลสถานะ

3.9.2 Message Parsing & Protocol

ใน Task รองรับเฟรมข้อมูลแบบไม่หน่วงซีพียู (Blocking Read) เมื่อมีเฟรมข้อมูลเข้ามา โปรแกรมจะตรวจสอบรหัสประจำตัว (Identifier หรือ ID) ของข้อความ และทำการแยกวิเคราะห์ข้อมูลในระดับไบนารีและบิต (Bitwise Operation) ตามมาตรฐานที่กำหนดไว้ในไฟล์เฮดเดอร์ดังนี้

1. ID 0x245 (ระบบปรับอากาศ - HVAC)
2. ID 0x325 (ระบบกระจกไฟฟ้า - WINDOW)
3. ID 0x420 (ระบบไฟห้องโดยสาร - DOMELIGHT)

3.10 ระบบแสดงผล (TFT Display & UI Management)

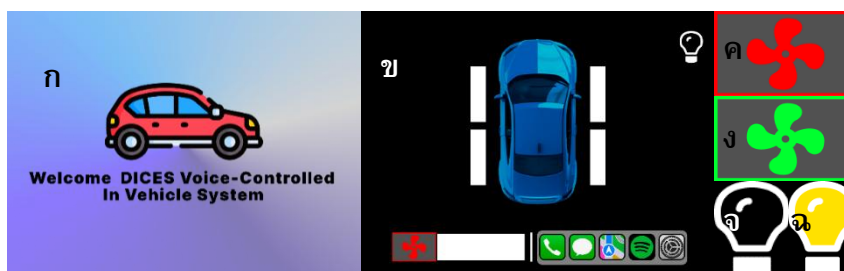
รับหน้าที่บริหารจัดการหน้าจอขนาด 3.2 นิ้วผ่านโปรโตคอล SPI โดยมีไลบรารี TFT_eSPI เป็นตัวขับเคลื่อน ภายใน Task Display มีการออกแบบลอจิกที่มีความเป็นไปได้

3.10.1 การอัปเดตเฉพาะจุด (Delta Rendering)

ลดการกระตุกของการวาดภาพ (Flicker) Task จะทำการเปรียบเทียบค่าข้อมูลสถานะปัจจุบัน กับค่าข้อมูลเดิมที่เคยแสดงผลไว้ หากข้อมูลไม่มีการเปลี่ยนแปลง โปรแกรมจะข้ามการวาดรูปส่วนนั้นไป ทำให้ประหยัดทรัพยากรซีพียูเป็นอย่างมาก

3.10.2 การวาดกราฟิกและแอนิเมชัน

หน้าต่าง UI ถูกออกแบบให้แบ่งเป็น กล้อง ตามพิกัด X, Y บนหน้าจอ เช่น กรอบสำหรับกระจกทั้ง 4 บาน และใช้การวาดรูปสี่เหลี่ยมเพื่อจำลองระดับกระจก รวมถึงมีการเรียกใช้งานไฟล์รูปภาพจากหน่วยความจำแฟลช (PROGMEM) เช่น มาวางทับ (Push Image) ในตำแหน่งที่กำหนดดังนี้



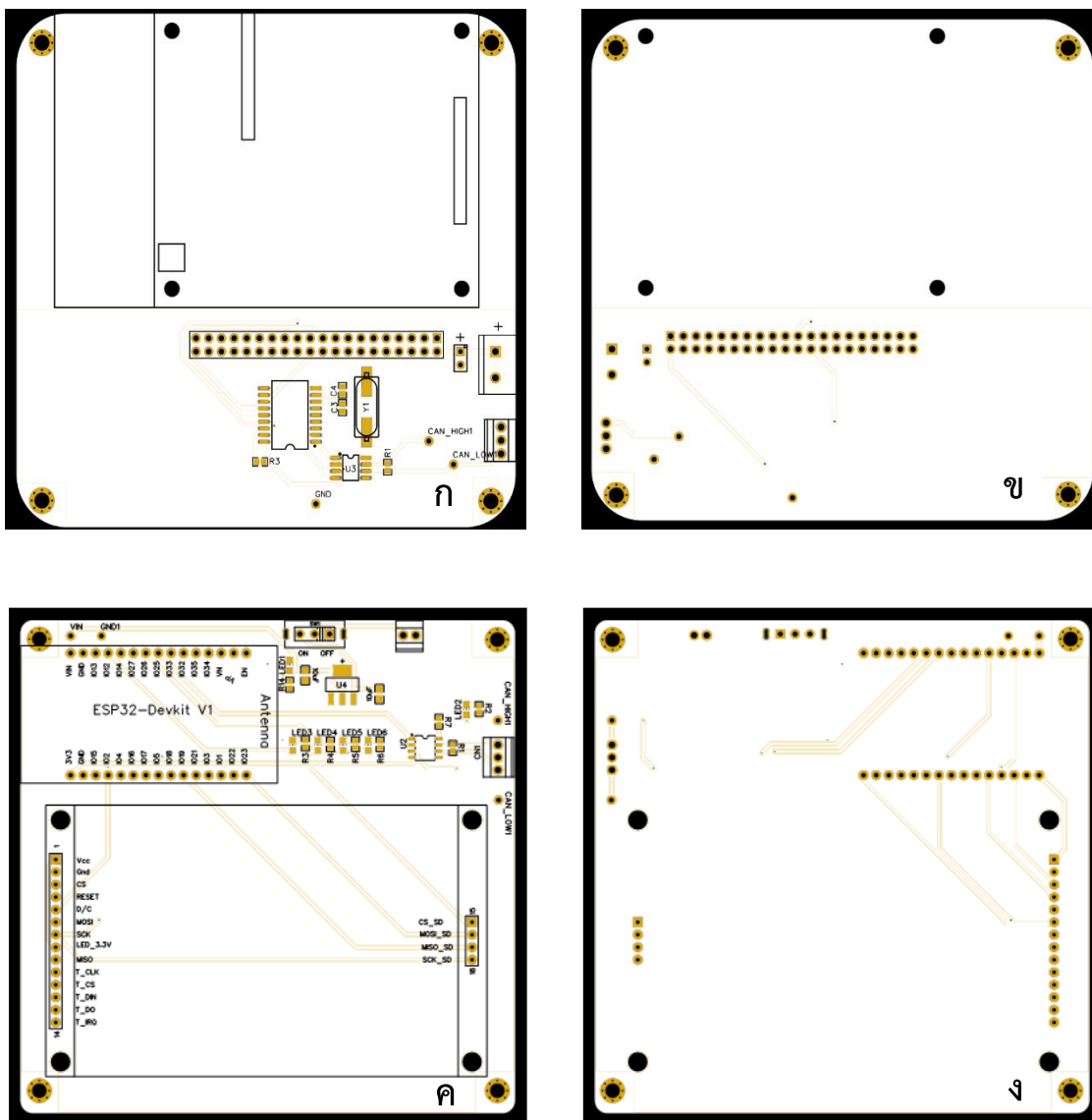
รูปที่ 19 กราฟิก

- (ก) รูปเปิดระบบของหน้าจอ; (ข) รูปโครงสร้างการแสดงผล; (ค) รูปการแสดงผลสำหรับการปิดแอร์; (ง) รูปโครงสร้างการแสดงผลการเปิดแอร์; (จ) รูปการแสดงผลสำหรับการปิดไฟรถยนต์; (ฉ) รูปการแสดงผลสำหรับการเปิดไฟรถยนต์

3.3.3 ควบคุมฮาร์ดแวร์ระดับรีจิสเตอร์ (Register-Level Hardware Control)

เมื่อได้รับสถานะระบบไฟส่องสว่างในห้องโดยสาร (Dome Light) โค้ดในส่วนนี้มีการสั่งงานเปิด-ปิดหลอดไฟ LED ที่ต่อกับขา GPIO โดยใช้คำสั่งระดับรีจิสเตอร์

3.4 PCB Layout

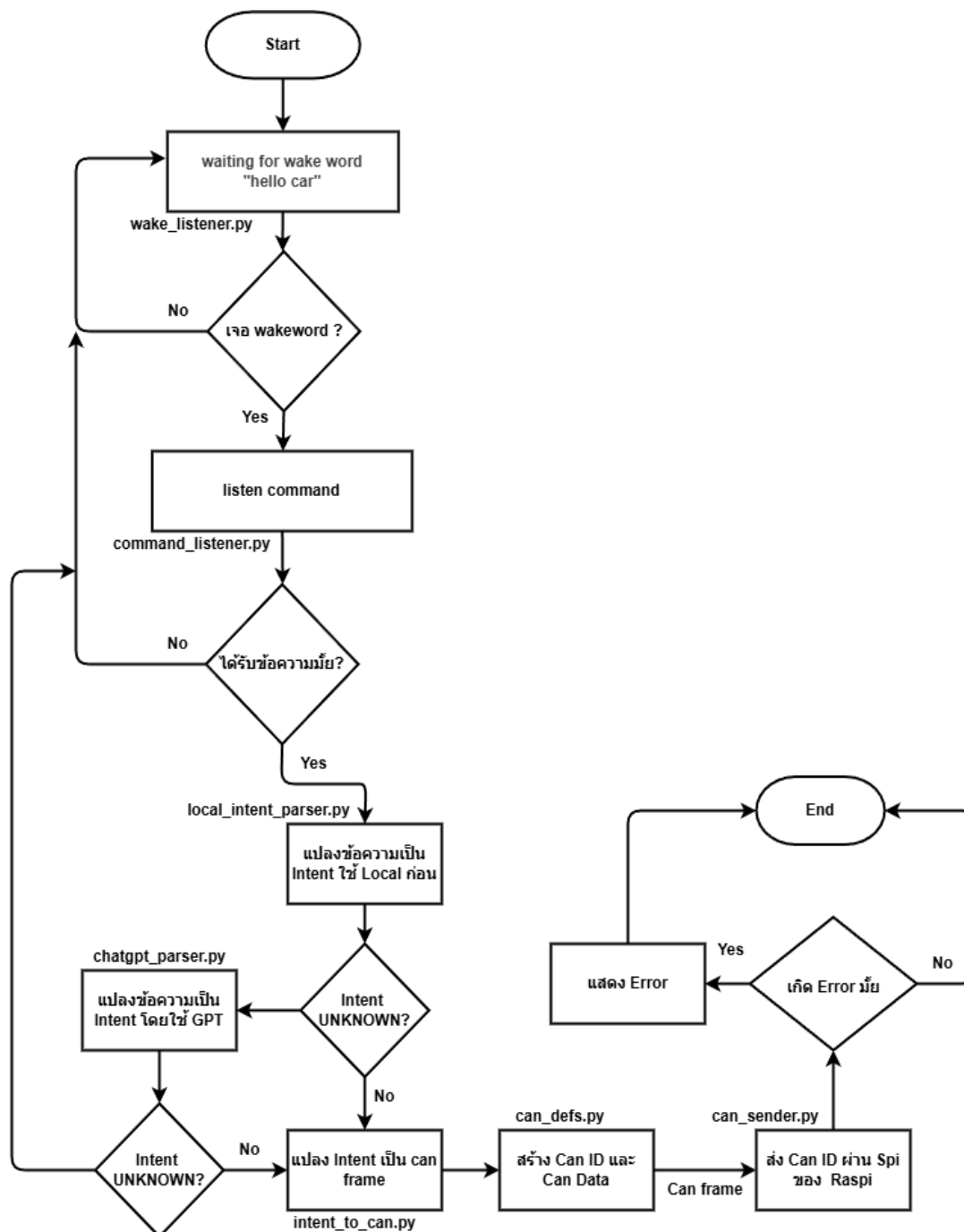


รูปที่ 20 แผนผัง PCB

(ก) PCB ภาคฝั่งส่ง Top layer; (ข) PCB ภาคฝั่งส่ง Bottom layer;
(ค) PCB ภาคฝั่งรับ Top layer; (ง) PCB ภาคฝั่งรับ Bottom layer

3.5 Flowchart software การทำงานของระบบ

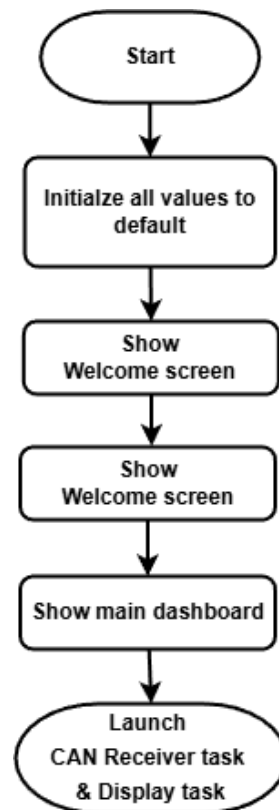
3.5.1 ภาคฟังการประมวลผลอินพุต และ การส่งข้อมูล



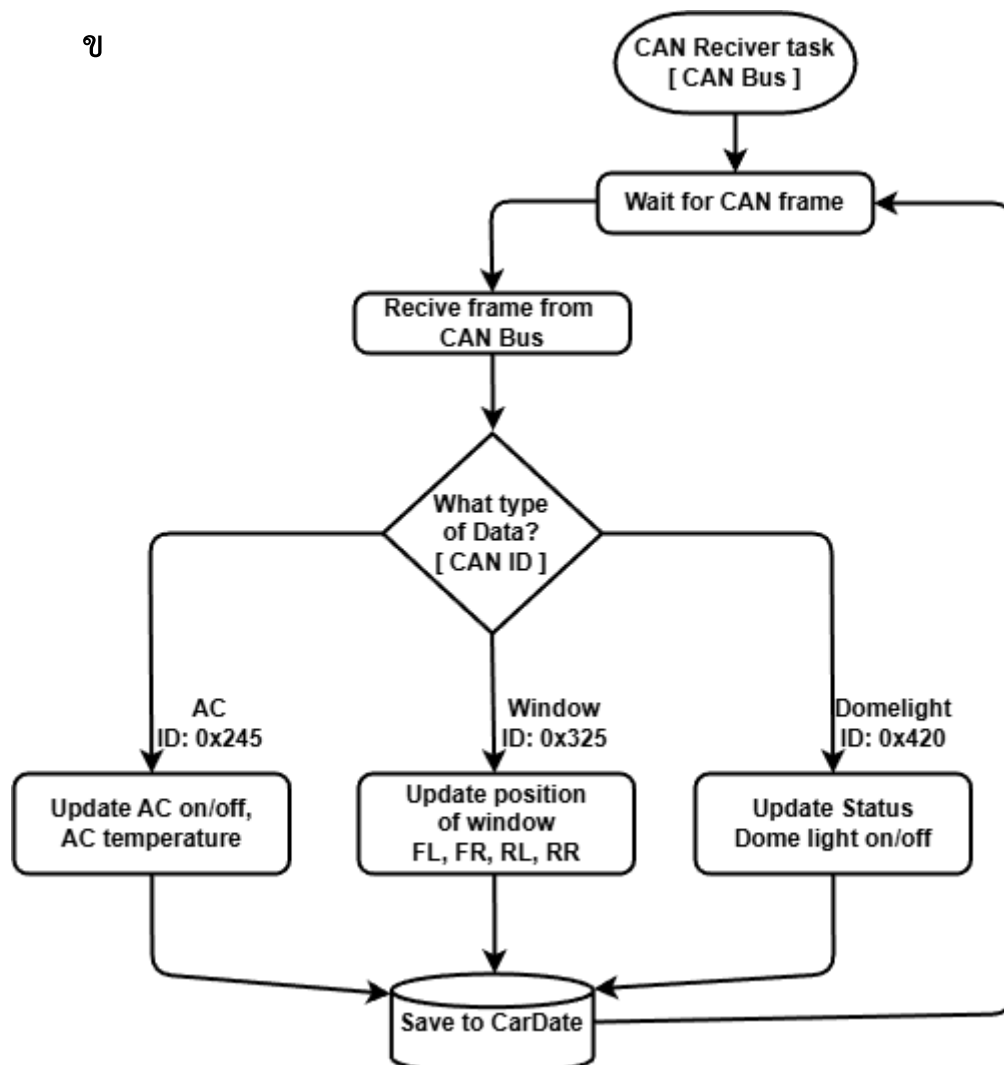
รูปที่ 21 FlowChart software ภาคฟังการประมวลผลอินพุต และ การส่งข้อมูล

3.5.2 ภาคฟังก์ชันการแสดงผลเอาต์พุต และ การรับข้อมูล

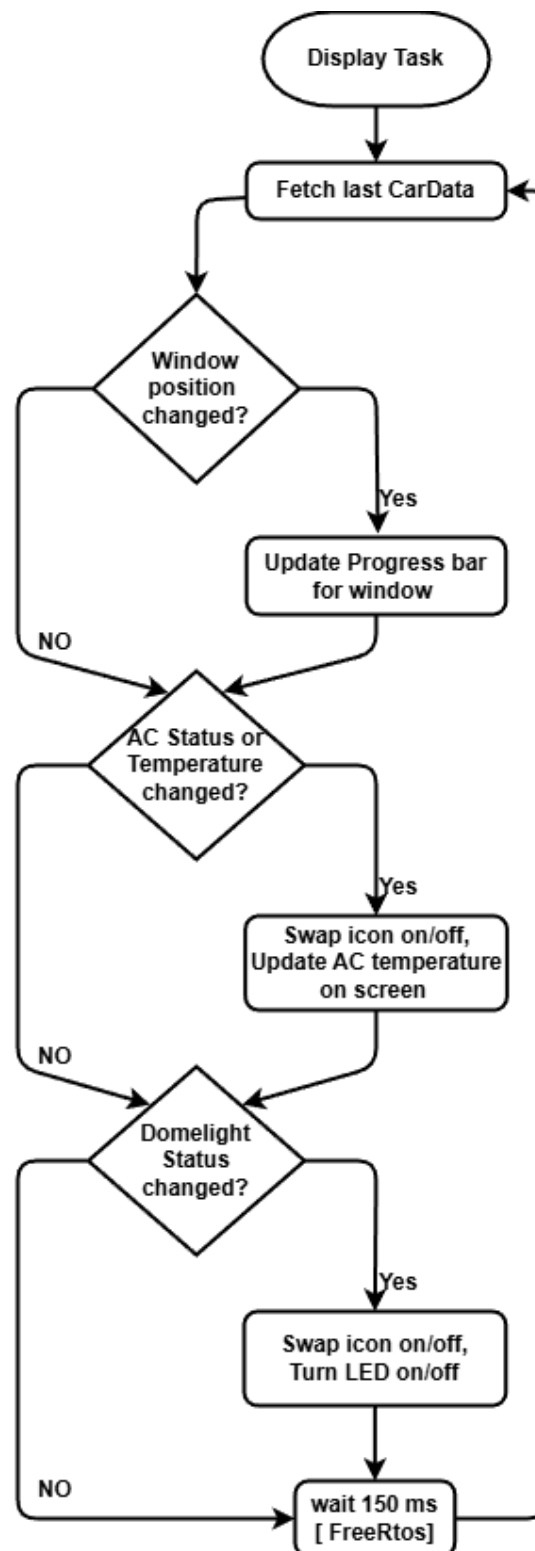
ก



ข



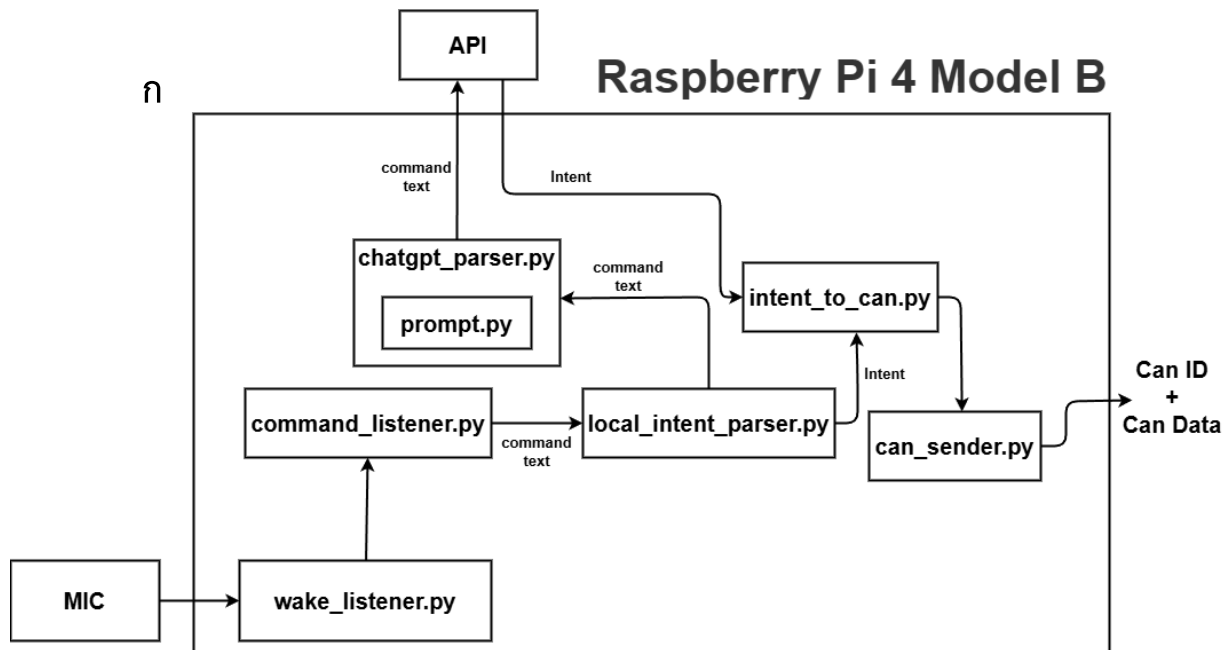
ค



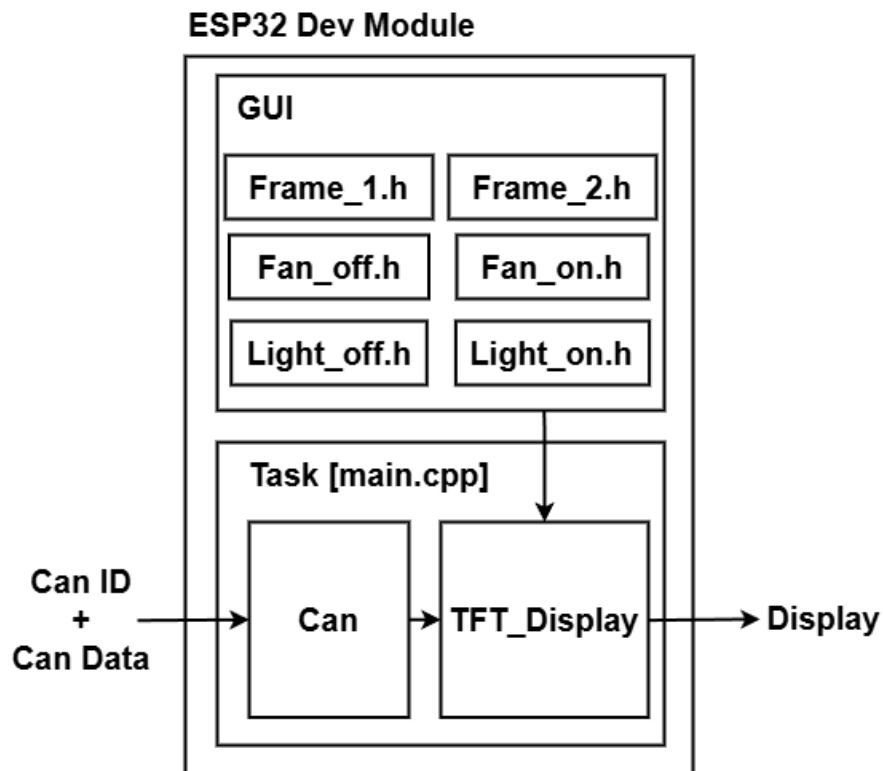
รูปที่ 22 Flowchart software ภาควงการแสดงผลเอาต์พุต และการรับข้อมูล

- ก) Flowchart function main
- ข) Flowchart function CAN
- ค) Flowchart function display

3.6 Block diagram



ข



รูปที่ 23 Block diagram

- ก) Block diagram ภาคการประมวลผลอินพุต และการส่งข้อมูล
 ข) Block diagram ภาคฝั่งการแสดงผลเอาต์พุต และการรับข้อมูล

บทที่ 4

ผลการทดลอง

4.1 ทดลองหน้าจอการเปิดระบบ



รูปที่ 24 ทดลองหน้าจอการเปิดระบบ

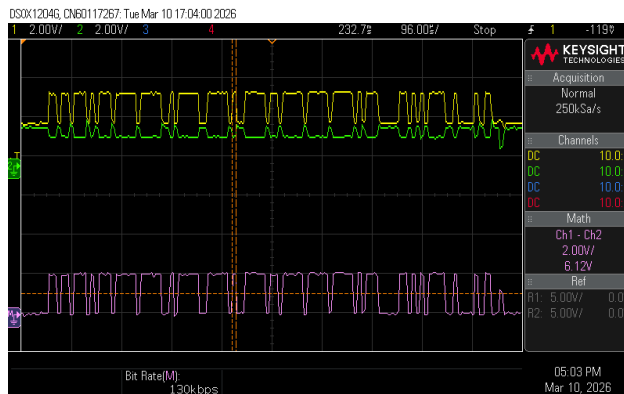
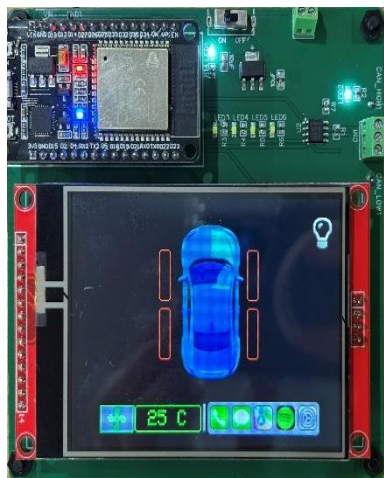
4.2 ทดลองหน้าจอแสดงผลของรถยนต์



รูปที่ 25 ทดลองหน้าจอแสดงผลของรถยนต์

4.3 ทดลองคำสั่งเปิดแอร์

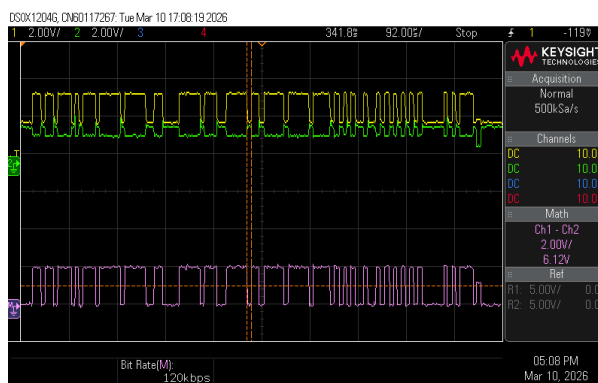
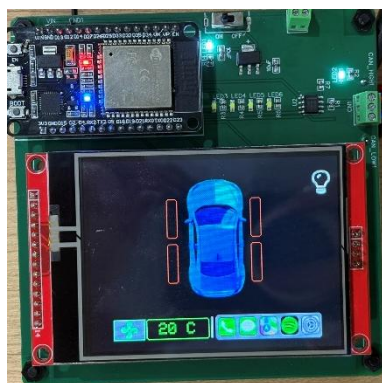
ID: 0x245 Data: 01 41 03 01 00 00 8F AA



รูปที่ 26 หน้าจอทดลองคำสั่งเปิดแอร์ และสัญญาณทดลองคำสั่งเปิดแอร์

4.4 ทดลองคำสั่งปรับแอร์เป็นอุณหภูมิ 20 องศา

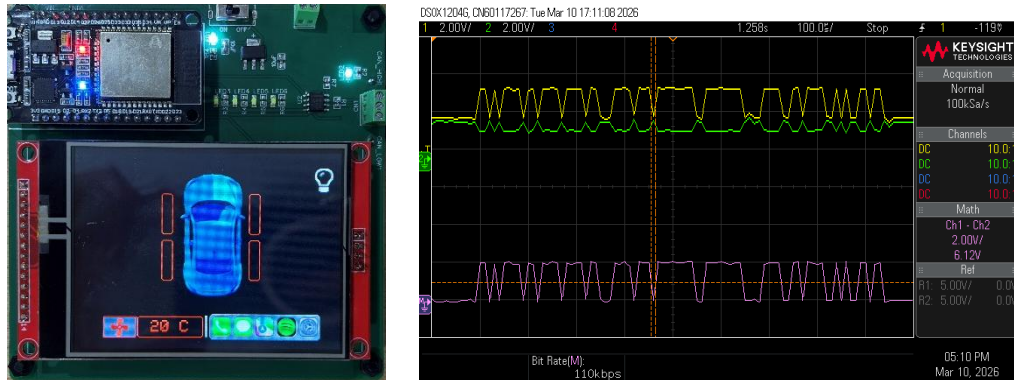
ID: 0x245 Data: 01 3C 03 01 00 02 AD AA



รูปที่ 27 หน้าจอทดลองคำสั่งปรับแอร์เป็นอุณหภูมิ 20 องศา และสัญญาณทดลองคำสั่งปรับแอร์เป็นอุณหภูมิ 20 องศา

4.5 ทดลองคำสั่งปิดแอร์

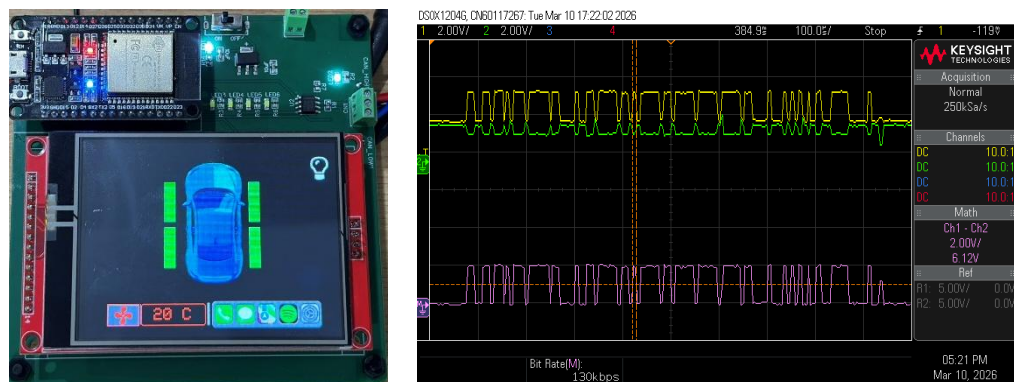
ID: 0x245 Data: 01 3C 03 00 00 06 56 AA



รูปที่ 28 หน้าจอทดลองคำสั่งปิดแอร์ และสัญญาณทดลองคำสั่งปิดแอร์

4.6 ทดลองคำสั่งเปิดกระจกทั้งหมด

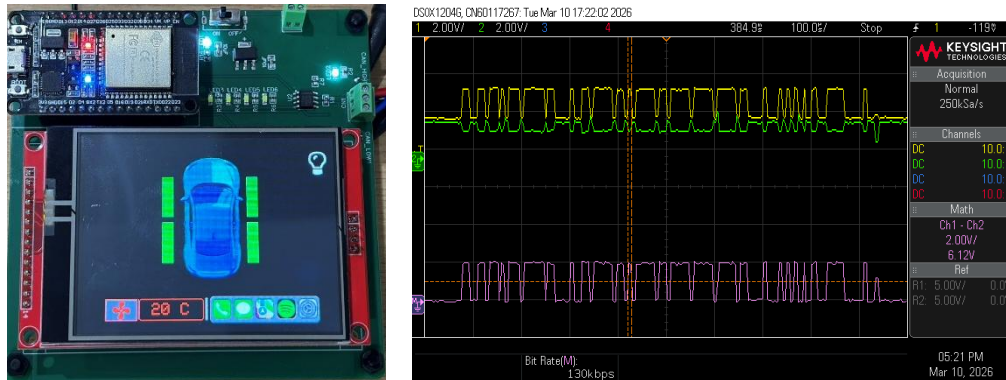
ID: 0x325 Data: 0F 64 05 01 00 07 28 AA



รูปที่ 29 หน้าจอทดลองคำสั่งเปิดกระจกทั้งหมด และสัญญาณทดลองคำสั่งเปิดกระจกทั้งหมด

4.6 ทดลองคำสั่งเปิดกระจกทั้งหมด

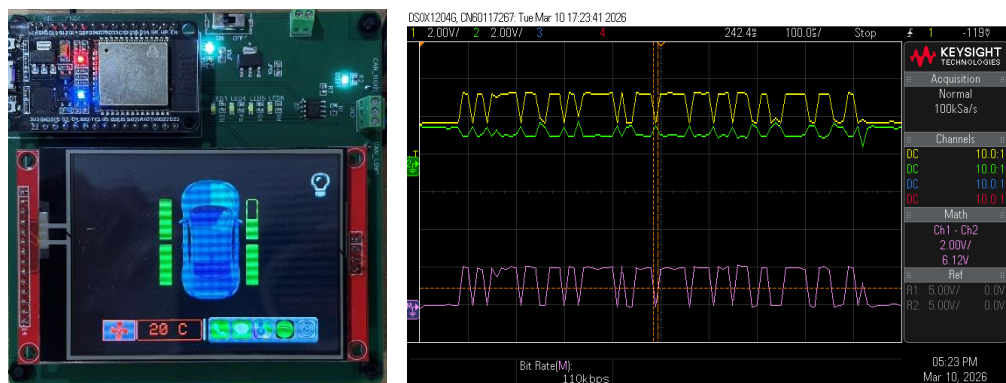
ID: 0x325 Data: 0F 64 05 01 00 07 28 AA



รูปที่ 30 หน้าจอทดลองคำสั่งเปิดกระจกทั้งหมด และสัญญาณทดลองคำสั่งเปิดกระจกทั้งหมด

4.7 ทดลองคำสั่งเปิดกระจกฝั่งคนขับครึ่งบาน

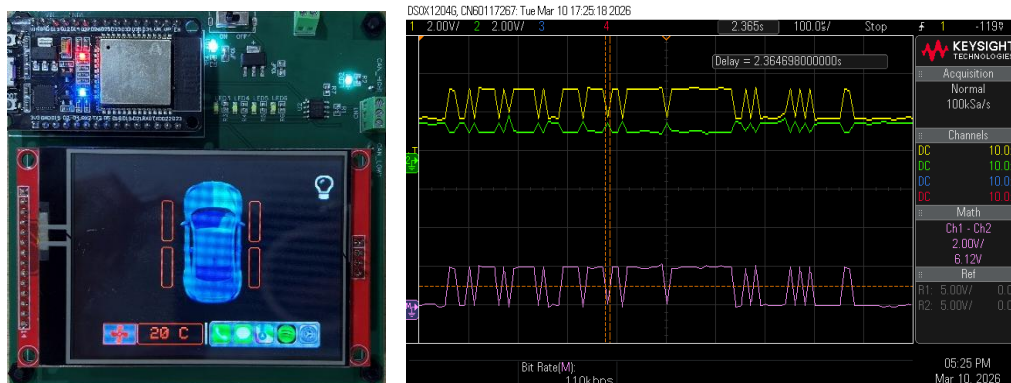
ID: 0x325 Data: 02 32 05 01 00 02 18 AA



รูปที่ 31 หน้าจอทดลองคำสั่งเปิดกระจกฝั่งคนขับครึ่งบาน และสัญญาณทดลองคำสั่งเปิดกระจกฝั่งคนขับครึ่งบาน

4.8 ทดลองคำสั่งปิดกระจกทั้งหมด

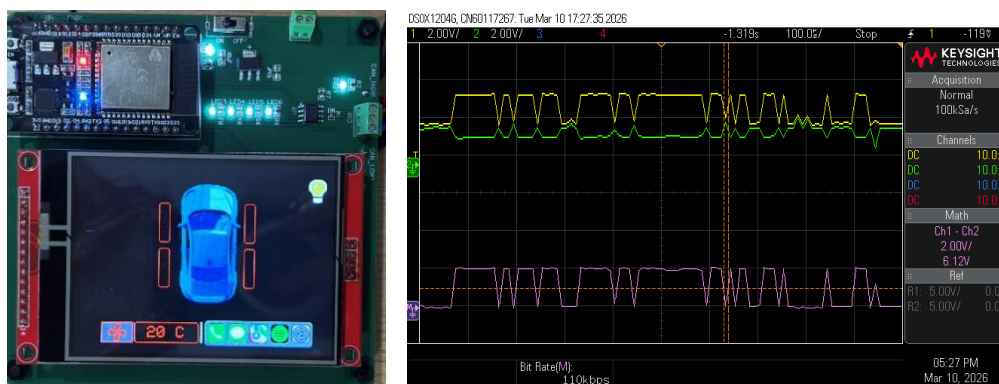
ID: 0x325 Data: 0F 00 05 01 00 03 B7 AA



รูปที่ 32 หน้าจอทดลองคำสั่งปิดกระจกทั้งหมด และสัญญาณทดลองคำสั่งปิดกระจกทั้งหมด

4.9 ทดลองคำสั่งเปิดไฟในรถ

ID: 0x420 Data: 01 64 00 00 00 04 D0 AA



รูปที่ 33 หน้าจอทดลองคำสั่งเปิดไฟในรถ และสัญญาณทดลองคำสั่งเปิดไฟในรถ

บทที่ 5

สรุปผลการทดลอง

5.1 สรุปภาพรวมผลการทำงานของระบบ

ระบบควบคุมภายในยานยนต์ด้วยเสียง ระบบรู้จำเสียง (Speech Recognition) เข้ากับระบบเครือข่ายสื่อสารจำลองภายในยานยนต์ ระบบสามารถรับคำสั่งเสียงจากผู้ใช้งาน วิเคราะห์ความต้องการ (Intent) ประมวลผลเป็นแพ็กเก็ตข้อมูล และส่งผ่านเครือข่าย CAN Bus (ความเร็ว 125 kbps) เพื่อควบคุมอุปกรณ์ปลายทางได้อย่างถูกต้องและตอบสนองแบบเรียลไทม์ จากการทดสอบส่งคำสั่งควบคุมอุปกรณ์ต่าง ๆ พบว่าระบบสามารถสร้างและส่ง CAN Message ได้ตรงตามโปรโตคอลที่ออกแบบไว้ การแสดงผลที่โหนดปลายทาง การประยุกต์ใช้ FreeRTOS บนไมโครคอนโทรลเลอร์ปลายทาง (เช่น ESP32) ช่วยให้การจัดการ Task ระหว่างการรับข้อมูลจาก CAN Bus และการแสดงผลสถานะบนหน้าจอ TFT ทำงานร่วมกันได้อย่างมีประสิทธิภาพ ไม่เกิดการสูญหายของข้อมูล (Data Loss)

5.2 ปัญหาและอุปสรรคที่พบ

ความแม่นยำของการรับคำสั่งเสียงอาจลดลงเมื่อนำไปใช้งานในสภาวะแวดล้อมที่มีเสียงรบกวน (Background Noise) ความหน่วง (Latency) ของระบบส่วนหนึ่งขึ้นอยู่กับความเร็วในการประมวลผลเสียง ซึ่งหากใช้ Cloud API จะต้องพึ่งพาสัญญาณอินเทอร์เน็ต

อ้างอิง

- [1] S. Corrigan, "Introduction to the Controller Area Network (CAN)," Application Report SLOA101B, Texas Instruments, 2016. [Online]. Available: <https://www.ti.com/lit/an/sloa101b/sloa101b.pdf>
- [2] Espressif Systems, "ESP32 Technical Reference Manual," 2023. [Online]. Available: <https://shorturl.asia/rZlBf>
- [3] Raspberry Pi Foundation, "Raspberry Pi 4 Model B Product Brief," 2019. [Online]. Available: <https://datasheets.raspberrypi.com/rpi4/raspberry-pi-4-product-brief.pdf>
- [4] FreeRTOS, "FreeRTOS Real Time Kernel Documentation." [Online]. Available: https://www.freertos.org/Documentation/RTOS_book.html
- [5] W. Voss, A Comprehensible Guide to Controller Area Network. Copperhill Media Corporation, 2008. [Online]. Available: <https://copperhillmedia.com/a-comprehensible-guide-to-controller-area-network/>
- [6] R. Barry, Using the FreeRTOS Real Time Kernel: A Practical Guide. Real Time Engineers Ltd., 2016. [Online]. Available: https://www.freertos.org/Documentation/RTOS_book.html
- [7] M. Eddin, "Feature extraction is one of the most important steps in developing a speech recognition system," 2020. [Online]. Available: <https://medium.com/@MuhyEddin/feature-extraction-is-one-of-the-most-important-steps-in-developing-a-speech-recognition-system-2a5b6f3b0f5b>
- [8] Rev.com, "What Is an Acoustic Model in Speech Recognition?" [Online]. Available: <https://www.rev.com/resources/what-is-an-acoustic-model-in-speech-recognition>
- [9] OpenAI, "OpenAI API Documentation: Text Generation," 2024. [Online]. Available: <https://platform.openai.com/docs/guides/text-generation>
- [10] A. Uberi, "SpeechRecognition: Library for performing speech recognition, with support for several engines and APIs, online and offline," 2023. [Online]. Available: <https://pypi.org/project/SpeechRecognition/>